CHAPTER 18

Engineering Foundations

ACRONYMS

CAD	Computer-Aided Design
CMMI	Capability Maturity Model Integration
PDF	Probability Density Function
PMF	Probability Mass Function
RCA	Root Cause Analysis
SDLC	Software Development Life Cycle

INTRODUCTION

The Institute of Electrical and Electronics Engineers (IEEE) defines engineering as "the application of a systematic, disciplined, quantifiable approach to structures, machines, products, systems or processes" [1]. As the theory and the practice of software engineering mature, it is increasingly apparent that software engineering as a discipline is based on skills and knowledge that are common to all engineering disciplines. This knowledge area (KA) explores engineering foundations pertinent to other engineering disciplines that also apply to software engineering. The focus is on covering topics that support other KAs while minimizing duplication of content covered elsewhere in this *Guide*.

BREAKDOWN OF TOPICS FOR ENGINEERING FOUNDATIONS

The breakdown of topics for the Engineering Foundations KA is shown in Figure 18.1.

1. The Engineering Process [2*, c4]

The engineering process, which is common to all engineering disciplines, is discussed more fully in the Software Engineering Economics KA. A brief, high-level summary is included here. Figure 18.2 shows the process flow.

The engineering process is necessarily iterative; knowledge gained at any point may be relevant to earlier steps, triggering iteration. These steps are briefly defined below:

- Understand the real problem — Engineering begins when a need is recognized and no existing solution meets that need. However, the problem that needs to be solved is not always the problem engineers are asked to solve. Use root cause analysis techniques (discussed later in this KA) to discover the underlying problem needing a solution.
- Define the selection criteria — Engineering decisions must consider various factors; for example, they must consider financial criteria, as discussed in the Software Engineering Economics KA. Be sure to identify all relevant selection criteria.
- Identify all reasonable, technically feasible solutions — The best solution is rarely the first solution that comes to mind. Therefore, consider multiple technically feasible solutions to ensure that the optimal solution is among the set considered.
- Evaluate each solution against the selection criteria — Determine how well each technically feasible solution satisfies the need while meeting the various criteria (for example, financial criteria).
- Select the preferred option — Identify which technically feasible solution best satisfies the selection criteria.
- Monitor the performance of the selected

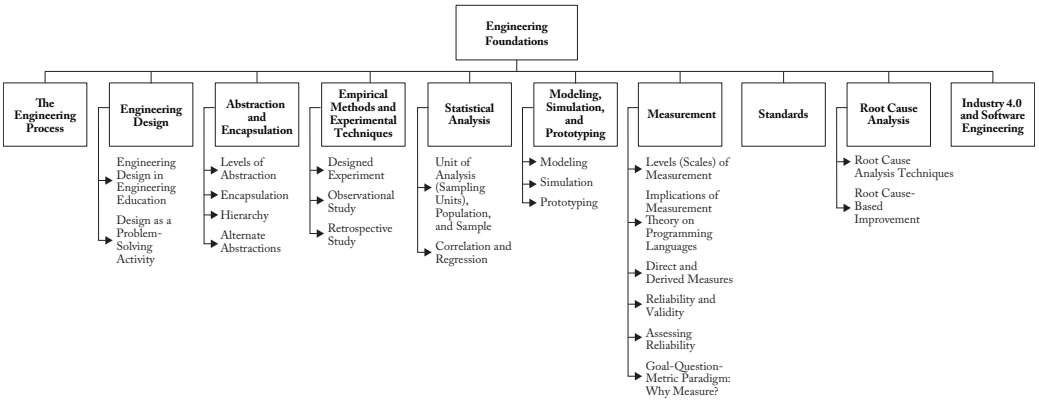

Figure 18.1. Breakdown of Topics for the Engineering Foundations KA

solution — The engineering process necessarily depends on estimates, and those estimates can be wrong. Therefore, it is essential to evaluate the selected alternative's real-world performance and, if necessary (and possible), decide whether one of the other alternatives might be better.

Much of the rest of this KA elaborates on details of this higher-level engineering process.

2. Engineering Design [3*, c1s2-s4]

A product's design will affect or even determine its life cycle costs. This is true for manufactured products as well as for software. Software design is guided by the features to be implemented and the quality attributes to be achieved. In the software engineering context, "design" has a particular meaning; while there are commonalities between engineering design as discussed in this section and software engineering design as discussed in the Software Architecture KA and the Software Design KA, there are also many differences. For example, the scope of engineering design is generally viewed as much broader than that of software design.

Many disciplines involve solving problems for which there is a single correct solution. In engineering, most problems have many solutions, and the focus is on finding a feasible solution (among many alternatives) that best meets the needs presented, economically. In business, where the goal may be to foster innovation in the marketplace, product definitions may derive from a business case. Whichever is the origin, possible solutions are often constrained by explicitly imposed limitations such as cost, available resources, and the state of discipline or domain knowledge. In engineering problems, implicit constraints (such as the physical properties of materials or the laws of physics) sometimes restrict the set of feasible solutions for a given problem.

2.1. Engineering Design in Engineering Education

Various engineering education accreditation bodies, including the Canadian Engineering Accreditation Board and the Accreditation Board for Engineering and Technology (ABET), place great value on engineering design, as evidenced by their high expectations in this area.

The Canadian Engineering Accreditation Board requires specified levels of engineering design experience and coursework for engineering students and certain qualifications for the faculty members who teach such coursework or supervise design projects. The organization's accreditation criteria state:

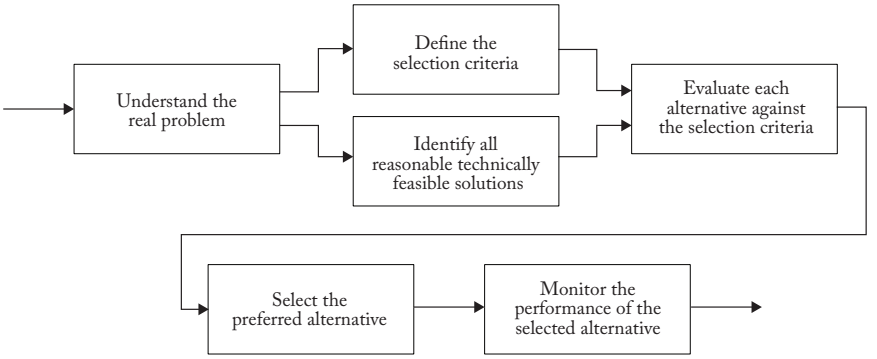

Figure 18.2. The Engineering Process

Design: An ability to design solutions for complex, open-ended engineering problems and to design systems, components or processes that meet specified needs with appropriate attention to health and safety risks, applicable standards, and economic, environmental, cultural, and societal considerations [4, p7].

Similarly, ABET defines engineering design as follows:

… a process of devising a system, component, or process to meet desired needs and specifications within constraints. It is an iterative, creative, decision-making process in which the basic sciences, mathematics, and engineering sciences are applied to convert resources into solutions [5, p7].

Thus, engineering design is vital to the training and education of all engineers. The rest of this section focuses on various aspects of engineering design.

2.2. Design as a Problem-Solving Activity [3*, c1s4, c2s1, c3s3] [6*, c5s1]

Engineering design is primarily a problem-solving activity. Finding a solution is particularly challenging because design problems tend to be open-ended and vaguely defined, and there are usually several ways to solve the same problem. Design is generally considered a *wicked problem* — a term coined by Horst Rittel in the 1960s when design methods were a subject of intense interest. Rittel sought an alternative to the linear, step-by-step process many designers and design theorists were exploring and argued that most problems addressed by designers are wicked problems. As explained by McConnell, a wicked problem presents a paradox: One can define it only by solving it, or by solving part of it. However, that solution is not the final solution; a wicked problem must be solved once to define it clearly and solved again to create a solution that works. This has been an important insight for software designers for decades [6*, c5s1].

3. Abstraction and Encapsulation [6*, c5s2-4]

Abstraction is an indispensable technique associated with problem-solving. It refers to both the process and the result of generalization, where one reduces the information about a concept, problem or observable phenomenon in order to focus on the "big picture." One of the most important skills in any engineering undertaking is the ability to frame the levels of abstraction appropriately.

According to Voland, "Through abstraction, we view the problem and its possible solution paths from a higher level of conceptual understanding. As a result, we may become better prepared to recognize possible relationships between different aspects of the

problem and thereby generate more creative design solutions" [2*]. This is true in computer science in general (such as hardware vs. software) and in software engineering in particular (e.g., data structure vs. data flow).

Dijkstra states, "The purpose of abstracting is not to be vague, but to create a new semantic level in which one can be absolutely precise" [7].

3.1. Levels of Abstraction

When abstracting, we concentrate on one "level" of the big picture at a time, confident that we can connect effectively with levels above and below. Although we focus on one level, abstraction does not mean knowing nothing about the neighboring levels. Abstraction levels do not necessarily correspond to discrete components in reality or in the problem domain, but to well-defined standard interfaces such as application programming interfaces (APIs). Standard interfaces offer advantages such as portability, easier software/hardware integration and wider usage.

3.2. Encapsulation

Encapsulation is a mechanism used to implement abstraction. When we are working with one level of abstraction, the information concerning the levels below and above that level is encapsulated. This can be information about the concept, problem, or observable phenomenon or the permissible operations on these entities. Encapsulation usually means hiding underlying details about the level above the interface provided by the abstraction. For example, hiding information about an object is useful because we don't need to know the details of how the object is represented or how the operations on the object are implemented.

3.3. Hierarchy

When we use abstraction in our problem formulation and solution, we might use different abstractions at different times — in other words, we work on different levels of abstraction as the situation requires. Usually, these different levels of abstraction are organized in a hierarchy. There are many ways to structure a particular hierarchy, and the criteria used in determining the specific content of each layer vary depending on the individuals performing the work.

Sometimes, a hierarchy of abstraction is sequential, meaning that each layer has one and only one predecessor (lower) layer and one and only one successor (upper) layer — except the upmost layer (which has no successor) and the bottommost layer (which has no predecessor). Sometimes, however, the hierarchy is organized in a tree structure, which means each layer can have more than one predecessor layer but only one successor layer. Occasionally, a hierarchy can have a many-to-many structure, in which each layer has multiple predecessors and successors. A hierarchy never contains a loop.

A hierarchy often forms naturally in task decomposition. Often, task analysis can be decomposed hierarchically, starting with the organization's larger tasks and goals and breaking each into smaller subtasks that can again be subdivided. This continuous division of tasks into smaller ones produces a hierarchical structure of tasks and subtasks.

3.4. Alternate Abstractions

Sometimes, multiple alternate abstractions for the same problem are useful to keep different perspectives in mind. For example, we can have a class diagram, a state chart and a sequence diagram for the same software at the same level of abstraction. These alternate abstractions do not form a hierarchy but complement each other, helping to illuminate the problem and its solution. Though beneficial, keeping alternate abstractions in sync is sometimes difficult.

4. Empirical Methods and Experimental Techniques [8*, c1]

The engineering process involves proposing solutions or models of solutions and

conducting experiments or tests to study those proposed solutions or models. Thus, engineers must understand how to create an experiment and analyze the results to evaluate proposed solutions. Empirical methods and experimental techniques help the engineer describe and understand variability in their observations, identify the sources of that variability, and make decisions.

Three types of empirical studies commonly used in engineering efforts are designed experiments, observational studies and retrospective studies. Brief descriptions of the commonly used methods are given below.

4.1. Designed Experiment

A *designed* or *controlled experiment* tests a hypothesis by manipulating one or more independent variables to measure their effect on one or more dependent variables. A precondition for conducting this experiment is the existence of a clear hypothesis. Therefore, engineers need to understand how to formulate clear hypotheses.

Designed experiments allow engineers to determine precisely how the variables are related and, specifically, whether a cause-effect relationship exists between them. Each combination of values of the independent variables is a *treatment*. The simplest experiments have just two treatments, representing two levels of a single independent variable (e.g., using a tool vs. not using a tool). More complex experimental designs arise when more than two levels, more than one independent variable, or any dependent variables are used.

4.2. Observational Study

An *observational* or *case study* is an empirical inquiry that makes observations of processes or phenomena within a real-world context. While an experiment deliberately ignores context, an observational or case study includes context. A case study is most useful when it focuses on *how* and *why* questions, on when the behavior of those involved cannot be manipulated, and on when contextual conditions are relevant and the boundaries between the phenomena and context are unclear.

4.3. Retrospective Study

Retrospective studies involve the analysis of historical data, and thus are also known as historical studies. This type of study uses data (regarding some phenomenon) archived over time. This archived data is then analyzed to find relationships between variables, to predict future events or to identify trends. One limitation is that the quality of the analysis depends on the quality of the archived data, and historical data may be incomplete, inconsistently measured or incorrect.

5. Statistical Analysis

[8*, c9s1, c2s1] [9*, c11s3]

Engineers must understand how product and process characteristics vary. Engineers often encounter situations where the relationship between different variables must be studied. Most studies use samples, but the results need to be understood with respect to the full population. Therefore, engineers must understand statistical techniques for collecting and interpreting reliable data (sampling and analysis) to arrive at results that can be generalized. These techniques are discussed below.

5.1. Unit of Analysis (Sampling Units), Population, and Sample

Unit of analysis. In any empirical study, the researchers must make observations based on chosen units called the units of analysis or sampling units. These units must be clearly identified and be appropriate for the analysis. For example, in a study of the perceived usability of a software product, the user or the software function might be the unit of analysis.

Population. The set of all respondents or items (possible sampling units) forms the population. For example, for a study of the perceived usability of a software product, the set of all possible users forms the population.

In defining the population, care must be taken to differentiate the study and target populations. The population being studied and the population for which the results are generalized will differ if the study involves a sample. For example, when the study population consists only of past observations but generalizations are required for the future, the study population and the target population are not the same.

Sample. A sample is a subset of the population. The most crucial issue in selecting a sample is its representativeness, including size. The samples must be drawn in a way that ensures draws are independent, and the rules of drawing samples must be predefined so the probability of selecting a particular sampling unit is known beforehand. This method of selecting samples is called *probability sampling*.

Random variable. In statistical terminology, the process of making observations or measurements on the sampling units is referred to as conducting the experiment. For example, if the experiment is to toss a coin 10 times and count the number of times the coin lands on heads, every 10 tosses of the coin is a sampling unit and the number of heads for a given sample is the observation or outcome for the experiment. The outcome of an experiment is obtained in terms of real numbers and defines the random variable being studied. The attribute of the items being measured at the outcome of the experiment represents the random variable being studied; the observation obtained from a particular sampling unit is a particular realization of the random variable. In the example of the coin toss, the random variable is the number of heads observed for each experiment.

The set of possible values of a random variable may be finite or infinite but countable (e.g., the set of all integers or the set of all odd numbers). In such a case, the random variable is called a *discrete random variable*. In other cases, the random variable under consideration may take values on a continuous scale and is called a *continuous random variable*.

Event. A subset of possible values of a random variable is called an event. Suppose X denotes some random variable; then, for example, we may define different events such as $X \geq x$ or $X < x$ and so on.

Distribution of a random variable. A random variable's range and pattern of variation are given by its distribution. When the distribution of a random variable is known, it is possible to compute the probability of any event. Some distributions occur commonly and are used to model many random variables occurring in practice in the context of engineering. A few of the more commonly occurring distributions are described below:

- *Binomial distribution* is used to model random variables that count the number of successes in *n* trials carried out independently of each other, where each trial results in success or failure. We assume that the chance of a successful trial remains constant [8*, c3s5].
- *Poisson distribution* is used to model the count of occurrences of some event over time or space [8*, c3s8].
- *Normal distribution* is used to model continuous or discrete random variables by taking a very large number of values [8*, c4s5].

Concept of parameters. Parameters characterize a statistical distribution. For example, the proportion of successes in any given trial is the only parameter characterizing a binomial distribution. Similarly, the Poisson distribution is characterized by a rate of occurrence. A normal distribution is characterized by two parameters: its mean and standard deviation.

Once the values of the parameters are known, the distribution of the random variable is revealed and the chance (probability) of any event can be computed. The probabilities for a discrete random variable can be computed through the probability mass function (PMF). The PMF is defined at discrete points and gives the point mass — i.e., the probability that the random variable takes that particular value. Likewise, for a continuous random variable, we have the probability density function (PDF). The PDF needs to be

integrated over a range to obtain the probability that the continuous random variable lies between certain values. Thus, if the PMF or PDF is known, the chances of the random variable taking a certain set of values may be computed theoretically.

Concept of estimation [8*, c7s1, c7s3]. The true values of the parameters of a distribution are usually unknown and need to be estimated from the sample observations. The estimates are functions of the sample values and are called *statistics*. For example, the sample mean is a statistic and may be used to estimate the population mean. Similarly, the rate of occurrence of defects estimated from the sample (rate of defects per line of code) is a statistic and serves as the estimate of the population rate of defects per line of code. The statistic used to estimate a population parameter is often referred to as the *estimator of the parameter*.

The results of the estimators themselves are random. If we take a different sample, we will likely get a different population parameter estimate. In the theory of estimation, we need to understand different properties of estimators — particularly, how much the estimates can vary across samples and how to choose between different ways to obtain the estimates. For example, if we wish to estimate the mean of a population, we might use as our estimator a sample mean, a sample median, a sample mode or the midrange of the sample. Each of these estimators has different statistical properties that might impact the standard error of the estimate.

Types of estimates [8*, c7s3, c8s1]. There are two types of estimates: *point estimates* and *interval estimates*. When we use the value of a statistic to estimate a population parameter, we get a point estimate. As the name indicates, a point estimate gives a point value of the parameter estimated.

Although point estimates are often used, they leave room for many questions. For instance, they do not tell us anything about the possible error size or the estimate's statistical properties. Thus, we might need to supplement a point estimate with information about the sample size and the estimate's variance. Alternatively, we might use an interval estimate. An interval estimate is a random interval whose lower and upper limits are functions of the sample observations and the sample size. The limits are computed based on assumptions regarding the sampling distribution of the point estimate on which the limits are based.

Properties of estimators. Various statistical properties of estimators are used to determine the appropriateness of an estimator in a given situation. The most important properties are efficiency, consistency with respect to the population, and lack of bias.

Tests of hypotheses [8*, c9s1]. A *hypothesis* is a statement about the possible values of a parameter. For example, suppose someone claims that a new method of software development reduces the occurrence of defects. The hypothesis is that the rate of occurrence of defects has been reduced. When we test the hypothesis, we decide — based on sample observations — whether it should be accepted or rejected.

To test hypotheses, the null and alternative hypotheses are formed. The *null hypothesis* is the hypothesis of no change, denoted as H_0. The *alternative hypothesis* is written as H_1. The alternative hypothesis may be one-sided or two-sided. For example, if we have the null hypothesis that the population mean is not less than some given value, the alternative hypothesis would be that it is less than that value, and we would have a one-sided test. However, if we have the null hypothesis that the population mean is equal to some given value, the alternative hypothesis would be that it is not equal, and we would have a two-sided test (because the true value could be either less than or greater than the given value).

The first step in testing a hypothesis is to compute a statistic. In addition, a region is defined such that if the computed value of the statistic falls within that region, the null hypothesis is rejected. This region is called the *critical region* (also known as the *confidence interval*). In tests of hypotheses, we need to accept or reject the null hypothesis based on the evidence obtained. In general, the alternative hypothesis is the hypothesis of interest.

If the computed value of the statistic does not fall inside the critical region, then we cannot reject the null hypothesis. This indicates that there is insufficient evidence to believe that the alternative hypothesis is true.

As the decision is based on sample observations, errors are possible; the types of such errors are summarized in the following table.

Nature	Statistical decision	
	Accept H_0	Reject H_0
H_0 is true	OK	Type I error (probability = α)
H_0 is false	Type II error (probability = β)	OK

In testing hypotheses, we aim to maximize the power of the test (the value of $1 - \beta$) while ensuring that the probability of a type I error (the value of α) is maintained within a particular value — typically 5%.

Also note that construction of a test of a hypothesis includes identifying statistic(s) to estimate the parameter(s) and defining a critical region such that if the computed value of the statistic falls within the critical region, the null hypothesis is rejected.

5.2. Correlation and Regression

[8*, c11s2, c11s8]

A major *objective* of many statistical investigations is to establish relationships that make it possible to predict one or more variables in terms of others. Although it is desirable to predict a quantity exactly in terms of another quantity, that is seldom possible, and, in many cases, we must be satisfied with estimating the average or expected values.

The relationship between two variables is studied using correlation and regression. Both these concepts are explained briefly below.

Correlation. The degree of the linear relationship between two variables is measured using the *correlation coefficient*. Computing the correlation coefficient is appropriate for two variables that measure two different attributes of the same entity. The correlation coefficient takes a value between –1 and +1. The values –1 and +1 indicate a situation where the association between the variables is perfect (i.e., given the value of one variable, the other can be estimated with no error). A positive correlation coefficient indicates a positive relationship (i.e., if one variable increases, so does the other). On the other hand, when the variables are negatively correlated, an increase of one leads to a decrease in the other.

Always remember that correlation does not imply causation. Thus, if two variables are correlated, we cannot conclude that one causes the other.

Regression. The correlation analysis only measures the degree of relationship between two variables. The analysis to find the strength of the relationship between two variables is called *regression analysis*. This analysis uses the coefficient of determination — a value between 0 and 1. The closer the coefficient is to 1, the stronger the relationship between the variables. A value of 1 indicates a perfect relationship.

6. Modeling, Simulation, and Prototyping

[3*, c6] [10*, c10s3] [11*, c5]

Modeling is part of the abstraction process used to represent aspects of a system. *Simulation* uses a model of the system to conduct designed experiments to better understand the system, its behavior and relationships among subsystems, as well as to analyze aspects of the design. Modeling and simulation can be used to construct theories or hypotheses about the system's behavior. Engineers then use those theories to make predictions about the system. *Prototyping* is another abstraction process where a partial representation (that captures aspects of interest) of the product or system is built. A prototype may be an initial version of the system that lacks the full functionality of the final version.

6.1. Modeling

A *model* is always an abstraction of some real or imagined artifact. Engineers use models in many ways as part of their problem-solving activities. Some models are physical, such as

a made-to-scale miniature construction of a bridge or building. Other models are non-physical representations, such as a computer-aided design (CAD) drawing of a cog or a mathematical model for a process. Models help engineers understand aspects of a problem. They can also help engineers determine what they know and what they don't know about the problem.

There are three types of models: iconic, analogic and symbolic. An *iconic model* is a visually equivalent but incomplete two-dimensional or three-dimensional representation (e.g., maps, globes or built-to-scale models of structures such as bridges or highways). An iconic model resembles the artifact modeled.

In contrast, an *analogic model* is a functionally equivalent but incomplete representation. The model behaves like the physical artifact even though it may not physically resemble it. Examples of analogic models include a miniature airplane for wind tunnel testing or a computer simulation of a manufacturing process.

Finally, a *symbolic model* uses a higher level of abstraction, modeling the process or system with symbols such as equations. The engineers can use the symbols to understand, describe, and predict the properties or behavior of the final system or product. An example is the equation F = ma.

6.2. Simulation

All simulation models are depictions of reality. A central issue in simulation is how to abstract data and create an appropriate simplification of reality. Developing this abstraction is vital, as misspecification of the abstraction would invalidate the results of the simulation exercise. Simulation can be used for a variety of testing purposes.

Simulation is classified based on the type of system under study; simulation can be either *continuous* or *discrete*. In software engineering, the emphasis is primarily on discrete simulation. Discrete simulations may model event scheduling or process interaction. The main components in such a model include entities, activities and events, resources, the state of the system, a simulation clock, and a random number generator. The simulation generates output that must be analyzed.

An important problem in the development of a discrete simulation is that of *initialization*. Before a simulation can be run, the initial values of all the state variables must be provided. As the simulation designer may not know what initial values are appropriate for the state variables, these values might be chosen somewhat arbitrarily. For instance, it might be decided that a queue should be initialized as empty and idle. This choice for an initial condition can have a significant but unrecognized impact on the simulation outcome.

6.3. Prototyping

Constructing a prototype of a system is another abstraction process. In this case, an initial version of the system is constructed, often while the system is designed, which helps the designers determine the feasibility of their design.

A prototype has many uses, including eliciting requirements, designing and refining a user interface, and validating functional requirements. The objectives and purposes for building the prototype will guide its construction and determine the level of abstraction used.

The role of prototyping is somewhat different for physical systems and software. With physical systems, the prototype might be the first fully functional version of a system, or it might be a model of the system. In software engineering, prototypes are also abstract models of part of the software. However, they are usually not constructed with all the architectural, performance and other quality characteristics expected in the finished product. In either case, prototype construction must have a clear purpose and be planned, monitored and controlled — it is a technique to study a specific problem within a limited context [12*, c2s8].

In conclusion, modeling, simulation and prototyping are powerful techniques for studying the behavior of a system from a

given perspective. All can be used to perform designed experiments to study various aspects of the system. However, these are abstractions and, as such, may not model all attributes of interest.

7. Measurement

[2*, pp442-447] [3*, c4s4] [12*, c7s5] [13*, c3s1-2]

Knowing what to measure, how to measure it, what can be done with measurements and even why to measure is critical in engineering endeavors. Everyone involved in an engineering project must understand the measurement methods, the measurement results and how those results can and should be used.

Measurements can be physical, environmental, economic, operational or another sort of measurement that is meaningful to the project. This section explores the theory of measurement and how it is fundamental to engineering. Measurement starts as an abstract concept and progresses to a definition of the measurement method and then to the actual application of that method to obtain a measurement result. Each step must be understood, communicated and properly performed to yield usable data. In traditional engineering, direct measures are often used. In software engineering, a combination of both direct and derived measures (defined in 7.3 below) is necessary [13*, p273].

The theory of measurement states that measurement is an attempt to describe an underlying empirical system. Measurement methods specify activities that assign a value or symbol to an attribute of an entity.

Attributes must then be defined in terms of the operations used to identify and measure them (the measurement methods). In this approach, a measurement method is defined as a precisely specified operation that yields a symbol (called the *measurement result*) as part of the measurement of an attribute. To be useful, the measurement method must be well defined. Arbitrariness or vagueness in the method leads to ambiguity in the measurement results.

In some cases — particularly in the physical world — the attributes we wish to measure are easy to grasp; however, in an artificial world like software engineering, defining attributes might not be that simple. For example, the attributes of height, weight, distance, etc., are easily and uniformly understood (though they may not be very easy to measure in all circumstances). In contrast, attributes such as software size and complexity require clear definitions.

Operational definitions. The definition of attributes, to start with, is often rather abstract. Such definitions do not facilitate measurements. For example, we might define a circle as a line forming a closed loop such that the distance between any point on this line and a fixed interior point called the center is constant. We might further say that the fixed distance from the center to any point on the closed loop is the circle's radius. Though the concept has been defined, no means of measuring the radius has been proposed. The operational definition specifies the exact steps or method used to carry out a specific measurement. This can also be called the *measurement method*; sometimes, a measurement procedure might be required to be even more precise.

The importance of operational definitions can hardly be overstated. Take the case of the apparently simple measurement of a person's height. Unless we specify various factors — for example, the time the height is measured (because the height of individuals varies throughout the day), how the variability created by hair is handled, whether the measurement is taken when the person is wearing shoes or not, the accuracy expected (to the nearest inch, 1/2 inch, centimeter, etc.) — then even this simple measurement will produce substantial variation. Therefore, engineers must appreciate the need to define measurements from an operational perspective.

7.1. Levels (Scales) of Measurement

[2*, pp442-447] [12*, c7s5] [13*, c3s2]

Once the operational definitions have been determined, actual measurements can be taken. Measurement may be carried out in

four different scale types: nominal, ordinal, interval, and ratio. Brief descriptions of each are given below:

Nominal scale: This is the lowest level of measurement and represents the most unrestricted assignment of symbols, which are only labels. Nominal scales involve classification where measured entities are put into one of the mutually exclusive and collectively exhaustive categories (classes). Examples of nominal scales are the following:

- Job titles in an organization
- Automobile styles (sedan, coupe, hatchback, minivan, etc.)
- Software development life cycle (SDLC) models (waterfall, iterative, Agile, etc.)

In nominal scales, no relationship among symbols may be inferred. The only valid types of manipulation of measures in a nominal scale are the following:

- Determining whether two entities have the same or different symbol (e.g., "Is your job title the same as or different from my job title?")
- Counting the number of entities having the same symbol (e.g., "How many employees have the job title Software Engineer Level 2 in this organization?")

Statistical analyses may be carried out to understand how entities belonging to different classes perform with respect to some other variable.

Ordinal scale: Ordinal scales extend nominal scales by requiring a strict ordering relationship among the symbols. Ordinal scales are necessarily transitive (if A > B and B > C, then A > C). The following are examples of ordinal scales:

- Finish order in a race (1st, 2nd, 3rd)
- Probabilities expressed using terms (remote, unlikely, even, probable, almost certain)
- Severities expressed using terms (negligible, marginal, serious, critical, catastrophic)
- Level of agreement expressed using terms (strongly agree, somewhat agree, neutral, somewhat disagree, strongly disagree)
- Capability Maturity Model Integration (CMMI) staged maturity levels

All manipulations of values on nominal scales are valid on ordinal scales, while ordinal scales also support more-than and less-than comparisons. For example:

- Did you finish that race before, tied with or after me?
- Is Event X the same, more or less probable than Event Y?
- Is Event X the same, more or less severe than Event Y?
- Is the CMMI staged maturity level of Organization A the same, higher or lower than that of Organization B?

When an ordinal scale uses numbers as symbols — like the CMMI staged maturity levels 1, 2, 3, 4 and 5 — those numbers cannot be manipulated arithmetically. We cannot say that the difference between CMMI staged maturity level 5 and level 4 (5 – 4) compares in any meaningful way to the difference between level 3 and level 2 (3 – 2). Neither can we say that CMMI staged maturity level 4 is twice as good as level 2. Ordinal scales that use numbers as symbols are commonly misused in exactly this way — for example, to present mean and standard deviation (e.g., "The average software organization worldwide has a CMMI staged maturity level of 1.763."). Such misuse can easily lead to erroneous conclusions [13*, p274]. (We can compute the median on an ordinal scale, as this only involves counting.) Using nonnumerical symbols, such as initial, repeatable, defined, managed, and optimizing (for CMMI staged maturity levels), is preferred because it helps prevent such mistreatment. Properly chosen labels also better communicate the meaning of each label.

Interval scale: Interval scales extend ordinal scales by requiring that the difference between any pair of adjacent values is constant. The following are examples of interval scales:

- Temperatures expressed in degrees Celsius and Fahrenheit: The difference between –9°C and –8°C is the same as that between 26°C and 27°C. The difference between –9°F and –8°F is the same as the difference between 26°F and 27°F.
- Calendar dates: The difference between any two consecutive dates is always one day: 24 hours.
- Shoe sizes in North America: The difference between size 3 and size 4 is the same as the difference between a size 10 and size 11 — one-third of an inch, or 8.467 mm.

All manipulations of values on ordinal scales are valid on interval scales, while interval scales also support addition and subtraction. For example:

- The difference between –9°C and 0°C is the same as that between 0°C and 9°C. The difference between –50°F and 0°F is the same as that between 25°F and 75°F.
- The length of time between May 6 and May 9 is the same as that between November 8 and November 11.

Interval scales support most statistical analyses, like mean, standard deviation, correlation and regression. Any manipulation involving multiplication or division of values, on the other hand, is meaningless because 0 on an interval scale, if it even exists, does not represent the absence of the measured quantity. A 0 point on an interval scale is arbitrary with respect to the attribute measured. Consider that 0° (both C and F) do not represent the absence of heat (absolute zero), and a North American size 0 shoe has non-zero length. Therefore, 30°C cannot be interpreted as twice as hot as 15°C, nor is a North American size 9 shoe three times longer than a size 3 shoe.

Ratio scale: Ratio scales extend ordinal scales by requiring the 0 point to represent the absence of the measured attribute. The following are examples of ratio scales:

- Temperature in degrees Kelvin (K)
- Shoe sizes in the Mondopoint system (commonly used for athletic shoes, ski boots, skates and ballet shoes); a size 270/105 shoe fits a foot 270 mm long and 105 mm wide
- Count of decision constructs (e.g., if(), for(), while(), in a given source code file)
- Money

Ratio scales support all arithmetic and statistical manipulations. Values in one ratio scale can often be trivially transformed into corresponding values in another ratio scale that measures the same attribute by using a multiplication factor. Distances in inches can be trivially transformed into centimeters, weights in kilograms can be trivially transformed into pounds, speed in knots can be trivially transformed into kilometers per hour, and so on.

An additional measurement scale, the *absolute scale*, is a ratio scale with uniqueness of measure (no transformation is possible). The number of software engineers working on a project is an absolute scale because there are no other meaningful measures for numbers of people.

7.2. Implications of Measurement Theory for Programming Languages

Common programming languages support a set of built-in data types, often including the following:

- Whole number types over varying ranges: int, integer, byte, short, long, etc.
- Floating-point numbers over varying ranges with varying precision: real, float, double, etc.
- Single characters: char
- Ordered sequences of characters: string

Many languages, although not all, also support type-safe enumeration (e.g., Java's "enum").

Unfortunately, these languages offer no support for measurement theory. They do not prevent, nor even warn programmers about, inappropriate manipulations. The whole and

floating-point number data types in programming languages operate as ratio scales and support the full range of manipulations: comparison, addition, subtraction, multiplication, division and so on. But consider CMMI staged maturity level expressed as a number. In measurement theory terms, as shown above, it is an ordinal scale, so addition, subtraction, multiplication and division are inappropriate. If any programmer represents a CMMI staged maturity level using a whole number data type, nothing prevents the programmer from inappropriately adding, subtracting, multiplying or dividing that number.

The same can be said for characters, strings and enumerations. Programming languages implement them as ordinal scales; however, they might only be intended for representing nominal-scale values. More-than and less-than comparisons are allowed even when inappropriate. The string "minivan" appears alphabetically before the string "sedan," but drawing any conclusion other than the mere alphabetical ordering of arbitrary text strings as a result of that fact is inappropriate.

Common programming languages allow programmers to easily write code that is inappropriate according to measurement theory. As long as programming languages allow it, programmers can and will — intentionally or unintentionally — misuse measurement scale types. A more sensible solution would be data-type semantics that explicitly enforce measurement theory. For example, a language could explicitly support nominal scales, as shown in Figure 18.3 Sample A. That language could then prevent, or at least warn programmers against, more-than or less-than comparisons as shown in Figure 18.3 Sample B.

If more-than or less-than comparisons are needed, the language supports declaration of an ordinal type as shown in Figure 18.3 Sample C. Figure 18.3 Sample D would not trigger any error or warning. Similarly, an interval scale could be supported, as shown in Figure 18.3 Sample E. A ratio scale could be supported, as shown in Figure 18.3 Sample F.

Common programming languages have no problem with the code shown in Figure 18.3 Sample G. On the other hand, a measurement theory–aware programming language would be expected to generate a compiler error or warning with the code shown in Figure 18.3 Sample H.

Future programming languages should enforce measurement theory and not allow developers to manipulate measurements inappropriately. But until languages support measurement theory, software engineers need to at least understand it and be on the lookout for inappropriate manipulations in, for example, code reviews.

7.3. Direct and Derived Measures

[13*, c7s5]

Measures may be either direct or derived (sometimes called *indirect measures*). An example of a *direct measure* is a count of how many times an event occurred, such as the number of defects found in a software product. A derived measure combines direct measures in a way that is consistent with the measurement methods used for those measures. For example, calculating the average hours spent to repair per defect is a derived measure. In both cases, the measurement method determines how to perform the measurement. The scale types of those measures constrain how they can be manipulated. When different scale types are involved:

- The scale type of the result of the manipulation can be no higher than the scale type of the most primitive measurement scale involved (e.g., a manipulation involving an interval scale and a ratio scale can only be done as if both are interval scales and can yield no better than an interval scale result).
- Investment is required to make the more primitive scale type compatible with any higher scale type (e.g., effort is required to bring the interval scale up to a ratio scale so the result can also be ratio-scaled).

```
nominal enum automobile_style = sedan, coupe, hatchback,
                                minivan, suv, sports_car;
```

Sample A

```
if( thisCarStyle >= sedan ) then … // this is not allowed
```

Sample B

```
ordinal enum CMMI_staged_level = initial, repeatable, defined,
                                 managed, optimizing;
```

Sample C

```
if( anOrgsCMMILevel > repeatable ) then …
```

Sample D

```
interval AirTemperatureCelsius from -120.0 to +180.0;
AirTemperatureCelsius yesterdaysHighTemp;
AirTemperatureCelsius todaysHighTemp;
if( todaysHighTemp > yesterdaysHighTemp ) { … } // allowed
if( todaysHighTemp > yesterdaysHighTemp * 2.0 ) { … } // not
```

Sample E

```
ratio TemperatureKelvin from 0.00 to 1000.00;
TemperatureKelvin previousReading;
TemperatureKelvin thisReading;
if( thisReading > previousReading * 2. ) { … } // allowed
```

Sample F

```
double priceOfBook;
double highTemperature;
highTemperature = priceOfBook; // makes no sense but is allowed
```

Sample G

```
ratio Money from -10000.00 to +10000.00;
ratio TemperatureKelvin from 0.00 to 1000.00;
Money priceOfBook;
TemperatureKelvin highTemperature;
double highTemperature;
highTemperature = priceOfBook; // not allowed
```

Sample H

Figure 18.3. Code Samples for Measurement Theory

7.4. Reliability and Validity [13*, c3s4-5]

A basic question to ask when considering any measurement method is whether the proposed measurement method is truly measuring the concept with good quality. Reliability and validity are the two most useful criteria for addressing this question.

The *reliability of a measurement method* is the extent to which the application of the method yields consistent results. Reliability refers to the consistency of the values obtained when the same item is measured several times. When the results agree with each other, the measurement method is said to be reliable. Reliability usually depends on the operational definition. It can be quantified by using the variation index, which is computed as the ratio between the standard deviation and the mean. The smaller the index, the more reliable the measurement results.

Validity refers to whether the measurement method measures what we intend to measure. The validity of a measurement method may be considered from three different perspectives: construct validity, criteria validity and content validity.

7.5. Assessing Reliability [13*, c3s5]

Methods for assessing reliability include the test-retest method, the alternative form

method, the split-halves method and the internal consistency method. The easiest of these is the test-retest method. In this method, we apply the measurement method twice to the same subjects. The correlation coefficient between the first and second set of measurement results gives us the reliability of the measurement method.

7.6. Goal-Question-Metric Paradigm: Why Measure?

The final concern to discuss here regarding measurement is the importance of understanding why we measure in the first place. The Goal-Question-Metric paradigm can be summarized with the simple observation that a measurement should be made to support decision-making. Some measurements support decisions in code. Other measurements support decisions made by people outside of code (e.g., process improvement measures). The critical point is that some decision should be made as a result of the measurement. Many real-world software organizations fall victim to a "measurement for the merely curious" syndrome, where metrics are gathered simply because they are easy to measure and interesting to look at when plotted in graphs. Those measurements are not used to support any decision and are a waste of time and energy. They should be avoided.

8. Standards [3*, c9s3.2]

Moore states that a standard can be the following:

a. An object or measure of comparison that defines or represents the magnitude of a unit
b. A characterization that establishes allowable tolerances for categories of items
c. A degree or level of required excellence or attainment

Standards are definitional in nature, established either to further understanding and interaction or to acknowledge observed (or desired) norms of exhibited characteristics or behavior [14, p8].

Standards provide requirements, specifications or guidelines that engineers must observe so that products, processes and materials are of acceptable quality. The qualities various standards dictate relate to safety, reliability or other product characteristics. Standards are considered critical to engineers, who are expected to be familiar with and use the appropriate standards for their specific discipline.

Compliance with or conformance to a standard allows an organization to assure the public that the organization's products meet the requirements contained in that standard. Thus, standards divide organizations or their products into those that conform to the standard and those that do not. For a standard to be useful, conformance must add real or perceived value to the product, process or effort.

Apart from supporting organizational goals, standards also serve several other purposes, such as protecting buyers, protecting businesses, and better defining the methods and procedures used in software engineering. Standards also provide users with common terminology and expectations.

There are many internationally recognized standards-making organizations, including the International Telecommunication Union (ITU), the International Electrotechnical Commission (IEC), IEEE, and the International Organization for Standardization (ISO). In addition, regional and governmentally recognized organizations generate standards for their region or country. For example, in the United States, more than 300 organizations develop standards. These include organizations such as the American National Standards Institute (ANSI), ASTM International (formerly known as American Society for Testing and Materials), SAE International (formerly the Society of Automotive Engineers), and Underwriters Laboratories, Inc. (UL), as well as the US government. (For more information on standards used in software engineering, see Appendix B.)

There is a set of commonly used principles behind standards. Standards makers attempt to reach consensus for their decisions. This approach fosters an openness within the community of interest so that once a standard is set, there is a good chance that it will be widely accepted. Most standards organizations have well-defined processes for their efforts and adhere to them carefully. Engineers must be aware of the existing standards and keep abreast of any changes to those standards over time.

In many engineering endeavors, understanding the applicable standards is critical, and the law may even require that specific standards be followed. In these cases, the standards often represent the minimal requirements that must be met and thus are an element of the constraints imposed on the design effort. Therefore, the engineer must review all current standards related to a given endeavor and determine which must be met. The design must then incorporate all constraints imposed by the applicable standard.

9. Root Cause Analysis [3*, c9s3-5] [13*, c5, c3s7, c9s8]

Root cause analysis (RCA) is a class of problem-solving methods for identifying underlying causes of undesirable outcomes. RCA methods identify why and how an undesirable outcome happened, allowing organizations to take effective action to prevent recurrence. Instead of merely addressing immediately obvious symptoms, the organization can solve problems by eliminating root causes. RCA can play several important roles in software projects, including the following:

1. Identifying the real problem to be solved by an engineering effort
2. Exposing the underlying drivers of risk, thus supporting project risk assessments
3. Revealing opportunities and actions for software process improvement
4. Discovering sources of recurring defects (defect causal analysis)

9.1. Root Cause Analysis Techniques

Several RCA techniques exist, including the following:

- *Change analysis* compares situations resulting in undesirable outcomes with similar situations that went well. The assumption is that the root cause will be found in the area of difference.
- The *5-whys* technique (see, for example, [2*, c4]) starts with an undesirable outcome and uses repeated "Why?" question-answer cycles to isolate the root cause.
- *Cause-and-effect diagrams*, sometimes called Ishikawa diagrams [15] or fishbone charts, break down, in successive levels of detail, causes that potentially contributed to an undesirable outcome. Causes are often grouped into major categories such as people, processes, tools, materials, measurements and environment. The diagram takes the form of a tree of potential causes that can all result in that undesirable outcome.
- *Fault tree analysis* (FTA) is a more formal approach to cause-and-effect diagramming that focuses on and/or relationships between causes and effects. In some cases, any one of multiple causes can drive the effect (an "or" relationship); in other cases, a combination of multiple causes is required to drive the effect (an "and" relationship). Cause-and-effect diagrams do not distinguish between *and* relationships and *or* relationships; fault tree analysis does.
- *Failure modes and effects analysis (FMEA)* forward-chains, starting with elements that can fail and cascade into undesirable effects. This approach contrasts with the backward-chaining techniques above, which start from an undesirable outcome and work backward toward causes.
- A *cause map* [16] is a structured map of cause-effect relationships that includes an undesirable outcome along with (1) chaining backward to driving causes and (2) chaining forward to effects on organizational goals. Cause maps require

evidence of the occurrence of causes and the causality of effects and are thus more rigorous than cause-and-effect diagrams, FTA, and FMEA.
- A *current reality tree* [17] is a cause-effect tree bound by the rules of logic (Categories of Legitimate Reservation).
- *Human performance evaluation* posits that human performance depends on (1) input detection, (2) input understanding, (3) action selection and (4) action execution. An undesirable outcome that results from human performance can be identified from a comprehensive list of potential drivers, including cognitive overload, cognitive underload (boredom), memory lapse, tunnel vision or lack of a bigger picture, complacency, and fatigue.

Additional techniques can be found in the DOE-NE-STD-1004-92 Root Cause Analysis Guidance Document.

9.2. Root Cause–Based Improvement

RCA is often an element in a greater process improvement effort. Why just identify a root cause if nothing will be done about it? Why go through the effort of identifying the root cause of low-importance problems? An example of a systematic process for a larger improvement effort incorporating RCA is given below:

1. Select the problem to solve: Techniques such as Pareto analysis (the "80/20 Rule"), frequency-severity prioritization (problems that happen most frequently and consume the most resources to rectify are the best candidates), and statistical process control are used to identify a high-priority, undesirable outcome to address. This step needs to clearly define the problem and its significance.
2. Gather evidence about that problem and its cause(s): Consider information surrounding the selected undesirable outcome, including statements or testimony, relevant processes or standards, specifications, reports, historical trends, experiments, or tests.
3. Identify the root cause using one or more RCA techniques presented in 9.1. Root Cause Analysis Techniques.
4. Select corrective action(s) that (1) prevent recurrence, (2) are within the organization's ability to control, (3) meet organizational goals and objectives, and (4) do not cause other problems. More than one candidate corrective action should be considered, and the potential actions should eliminate the cause, reduce the probability of the cause occurring or disconnect the cause from the effect. Selected corrective actions should generate the greatest amount of control for the least cost.
5. Implement the selected corrective action(s).
6. Observe the selected corrective action(s) to ensure efficiency and effectiveness.

10. Industry 4.0 and Software Engineering

The manufacturing industry has always been continuously changing. Industry 4.0 is set to change the manufacturing segment significantly, primarily focusing on custom manufacturing supported by artificial intelligence (AI). This offers potential benefits for cost, quality and efficiency. Industry 4.0's emphasis on digitization and AI calls for building bespoke hardware and software and integrating these with other standard systems.

This is supported by Continuous Software Engineering (CSE), which has been addressing continuous manufacturing practices such as continuous planning, continuous architecting/designing, continuous development, continuous integration, continuous deployments and continuous review/revision.

Software is a key component in the Industry 4.0 revolution, and engineering the software is crucial to building robust and intelligent systems. The engineering for one product affects others, as more devices connect with other devices, mostly wirelessly, to provide data and receive commands and data for further functioning.

Many technologies are used in Industry 4.0, including the Internet of Things (IoT), Big data analytics, AI and machine learning, cybersecurity, cloud computing and apps for multiple platforms among others. Software plays a key role in the implementation of all these.

Continuous Systems and Software Engineering for Industry 4.0 (CSSE I4.0) proposes how software engineering could be applied in Industry 4.0. Quantum computing enables complex computations to be performed faster and more cost-effectively. The size and cost of devices that host the software are decreasing significantly, easing the adoption of Industry 4.0. The software will be increasingly self-learning and proactive, developing the ability to predict users' wants.

MATRIX OF TOPICS VS. REFERENCE MATERIAL

	Tockey 2004 [2*]	**Voland 2003 [3*]**	**McConnell 2004 [6*]**	**Montgomery and Runger 2018 [8*]**	**Null and Lobur 2018 [9*]**	**Cheney and Kincaid 2020 [10*]**	**Sommerville 2016 [11*]**	**Fairley 2009 [12*]**	**Kan 2002 [13*]**
1. The Engineering Process	c4								
2. Engineering Design		c1s2-4							
2.1. Engineering Design in Engineering Education									
2.2. Design as a Problem-Solving Activity		c1s4, c2s1, c3s3	c5s1						
3. Abstraction and Encapsulation			c5s2-4						
3.1. Levels of Abstraction									
3.2. Encapsulation									
3.3. Hierarchy									
3.4. Alternate Abstractions									
4. Empirical Methods and Experimental Techniques				c1					
4.1. Designed Experiment									
4.2. Observational Study									
4.3. Retrospective Study									
5. Statistical Analysis				c9s1, c2s1	c11s3				

5.1. Unit of Analysis (Sampling Units), Population and Sample				c3s5, c3s8, c4s5, c7s1, c7s3, c8s1, c9s1					
5.2. Correlation and Regression				c11s2, c11s8					
6. Modeling, Simulation and Prototyping		c6				c10s3	c5		
6.1. Modeling									
6.2. Simulation									
6.3. Prototyping								c2s8	
7. Measurement	pp 442-447	c4s4						c7s5	c3s1-2
7.1. Levels (Scales) of Measurement	p442-447							c7s5	c3s2
7.2. Implications of Measurement Theory for Programming Languages									
7.3. Direct and Derived Measures									c7s5
7.4. Reliability and Validity									c3s4-5
7.5. Assessing Reliability									c3s5
7.6. Goal-Question-Metric Paradigm: Why Measure?									
8. Standards		c9s3.2							
9. Root Cause Analysis		c9s3-5							c5, c3s7, c9s8
9.1. Root Cause Analysis Techniques	c4								
9.2. Root Cause-Based Improvement									
10. Industry 4.0 and Software Engineering									

FURTHER READINGS

A. Abran, *Software Metrics and Software Metrology*. [18]

This book provides very good information on the proper use of the terms *measure*, *measurement method* and *measurement outcome*. It provides strong support material for the entire section on measurement.

W.G. Vincenti, *What Engineers Know and How They Know It*. [19]

This book introduces engineering foundations through case studies showing many foundational concepts in real-world engineering applications.

REFERENCES

[1] ISO/IEC/IEEE 24765:2017, Systems and Software Engineering — Vocabulary.

[2*] S. Tockey, *Return on Software: Maximizing the Return on Your Software Investment*, 1st ed., Addison-Wesley, 2004.

[3*] G. Voland, *Engineering by Design*, 2nd ed., Prentice Hall, 2003.

[4] "2021 Accreditation Criteria and Procedures," Canadian Engineering Accreditation Board, Engineers Canada, 2021.

[5] E.A. Commission, "Criteria for Accrediting Engineering Programs, 2022-2023," ABET, 2021.

[6*] S. McConnell, *Code Complete*, 2nd ed., Microsoft Press, 2004.

[7] Edsger W. Dijkstra, "The Humble Programmer," *Communications of the ACM*, vol. 15, issue 10, October 1972.

[8*] D.C. Montgomery and G.C. Runger, *Applied Statistics and Probability for Engineers*, 7th ed. Hoboken, NJ: Wiley, 2018.

[9*] L. Null and J. Lobur, *The Essentials of Computer Organization and Architecture*, 5th ed. Sudbury, MA: Jones and Bartlett Publishers, 2018.

[10*] E.W. Cheney and D.R. Kincaid, *Numerical Mathematics and Computing*, 7th ed. Belmont, CA: Brooks/Cole, 2020.

[11*] I. Sommerville, *Software Engineering*, 10th ed. New York: Addison-Wesley, 2016.

[12*] R.E. Fairley, *Managing and Leading Software Projects*. Hoboken, NJ: Wiley-IEEE Computer Society Press, 2009.

[13*] S.H. Kan, *Metrics and Models in Software Quality Engineering*, 2nd ed. Boston: Addison-Wesley, 2002.

[14] J.W. Moore, *The Road Map to Software Engineering: A Standards-Based Guide*, 1st ed. Hoboken, NJ: Wiley-IEEE Computer Society Press, 2006.

[15] K. Ishikawa, *Introduction to Quality Control*, Productivity Press, 1990.

[16] D. Gano, *Apollo Root Cause Analysis*, 3rd ed., Apollonian Publications, 2007.

[17] E. Goldratt, *It's Not Luck*, North River Press, 1994.

[18] A. Abran, *Software Metrics and Software Metrology*: Wiley-IEEE Computer Society Press, 2010.

[19] W.G. Vincenti, *What Engineers Know and How They Know It*. Johns Hopkins University Press, 1993.

[20] Elisa Yumi Nakagawa, Pablo Oliveira Antonio, Frank Schnicke, Thomas Kuhn, Peter Liggesmeyer, *Continuous Systems and Software Engineering for Industry 4.0: A disruptive view,* Elsevier, Volume 135, July 2021, 106562 (https://www.sciencedirect.com/science/article/abs/pii/S0950584921000458.)

KNOWLEDGE AREA DESCRIPTION SPECIFICATIONS

Appendix A

INTRODUCTION

This appendix presents the specifications provided to the knowledge area (KA) editors regarding the KA Descriptions of the *Guide to the Software Engineering Body of Knowledge, Version 4 (SWEBOK Guide, V4)*. This enables readers, reviewers and users to clearly understand what specifications were used in developing this version of the *SWEBOK Guide*.

This appendix begins by situating the *SWEBOK Guide* as a foundational document for the IEEE Computer Society's suite of software engineering products and more widely within the software engineering community. The appendix then describes the role of the baseline and change control. Criteria and requirements are defined for the breakdowns of topics, for the rationale underlying these breakdowns and the succinct description of topics, and for reference materials. Important input documents are also identified, and their role within the project is explained. Finally, non-content issues such as submission format and style guidelines are discussed.

THE SWEBOK GUIDE IS A FOUNDATIONAL DOCUMENT FOR THE IEEE COMPUTER SOCIETY SUITE OF SOFTWARE ENGINEERING PRODUCTS

The *SWEBOK Guide* is an IEEE Computer Society flagship and structural document for the IEEE Computer Society's suite of software engineering products. The *SWEBOK Guide* is also more widely recognized as a foundational document throughout the software engineering community, notably through the official recognition of the 2004 and 2014 versions as ISO/IEC Technical Report 19759:2005 and 19759:2015, respectively. The list of KAs and the breakdown of topics within each are described and detailed in this *SWEBOK Guide*'s introduction. Consequently, the *SWEBOK Guide* is foundational to other initiatives within the IEEE Computer Society, as follows:

- The list of KAs and the breakdown of topics within each are also adopted by the software engineering certification and associated professional development products offered by the IEEE Computer Society. (See www.computer.org/certification.)
- The list of KAs and the breakdown of topics are also foundational to the software engineering curriculum guidelines developed or endorsed by the IEEE Computer Society. (See https://www.computer.org/volunteering/boards-and-committees/professional-educational-activities/curriculum-accreditation-committee)
- The Consolidated Reference List (see Appendix C) — meaning the list of Recommended References (to the level of section number) that accompanies the breakdown of topics within each KA — is also adopted by the software engineering certification and associated professional development products offered by the IEEE Computer Society.

BASELINE AND CHANGE CONTROL

Due to the structural nature of the *SWEBOK Guide* and its adoption by other products, a baseline was developed at the outset of the project by a SWEBOK Steering Group. The

baseline comprises the list of KAs, including new ones, and the breakdown of topics for each KA from the previous version.

Furthermore, a *SWEBOK* KA editors team has been put in place for the development of this version to handle all major change requests to this baseline coming from the KA editors, arising during the review process or otherwise. Change requests must be approved both by the *SWEBOK Guide* editors and by the team before being implemented. The team is composed of members of the initiatives listed above and acts under the authority of the Engineering Discipline Committee of the IEEE Computer Society Professional and Educational Activities Board (PEAB).

CRITERIA AND REQUIREMENTS FOR THE BREAKDOWN OF TOPICS WITHIN A KNOWLEDGE AREA

- KA editors are instructed to refine the baseline breakdown of topics to reflect the recent development in the target area for KAs that continue to exist from the previous version.
- The breakdown of topics is expected to be "reasonable," not "perfect."
- The breakdown of topics within a KA must decompose the subset of the *SWEBOK* that is "generally recognized." (See below for a more detailed discussion of this point.)
- The breakdown of topics within a KA must not presume specific application domains, business needs, sizes of organizations, organizational structures, management philosophies, software life cycle models, software technologies or software development methods.
- The breakdown of topics must, as much as possible, be compatible with the various schools of thought within software engineering.
- The breakdown of topics within a KA must be compatible with the breakdown of software engineering generally found in industry and in the software engineering literature and standards.
- The breakdown of topics is expected to be as inclusive as possible.
- The *SWEBOK Guide* adopts the position that even though the following "themes" are common across all KAs, they are also an integral part of all KAs and, therefore, must be incorporated into the proposed breakdown of topics of each KA. These common themes are measurement, quality (in general) and security.
- The breakdown of topics should be at most two or three levels deep. Even though no upper or lower limit is imposed on the number of topics within each KA, a reasonable and manageable number of topics is expected to be included in each KA. Emphasis should also be put on the selection of the topics themselves rather than on their organization in an appropriate hierarchy.
- Topic names must be significant enough to be meaningful even when cited outside the *SWEBOK Guide*.
- The Description of a KA will include a chart (in tree form) describing the knowledge breakdown. This chart will typically be the first figure in the respective KA.

CRITERIA AND REQUIREMENTS FOR DESCRIBING TOPICS

Topics need only be sufficiently described so readers can select the appropriate reference material according to their needs. Topic descriptions must not be prescriptive.

CRITERIA AND REQUIREMENTS FOR REFERENCE MATERIAL

- KA editors are instructed to use the references (to the level of section number) allocated to their KA by the Consolidated Reference List as their Recommended References.
- There are three categories of reference material:
  - » Recommended References. The set of Recommended References (to the level

of section number) is collectively known as the Consolidated Reference List.
  - » Further Readings.
  - » Additional references cited in the KA Description (e.g., the source of a quotation or reference material in support of a rationale behind a particular argument).
- The *SWEBOK Guide* is intended by definition to be selective in its choice of topics and associated reference material. The list of reference material should be clearly viewed as an "informed and reasonable selection" rather than as a definitive list.
- Reference material can be book chapters, refereed journal papers, refereed conference papers, refereed technical or industrial reports, or any other type of recognized artifact. References to another KA, subarea or topic are also permitted.
- Reference material must be generally available and must not be confidential in nature.
- Reference material must be in English.
- Criteria and requirements for recommended reference material or Consolidated Reference List:
  - » Collectively, the list of Recommended References should be:
    i. Complete — covering the entire scope of the *SWEBOK Guide*
    ii. Sufficient — providing enough information to describe "generally accepted" knowledge
    iii. Consistent — not providing contradictory knowledge or conflicting practices
    iv. Credible — recognized as providing expert treatment
    v. Current — treating the subject in a manner that is commensurate with current, generally accepted knowledge
    vi. Succinct — as short as possible (both in the number of reference items and in total page count) without failing other objectives
  - » Recommended reference material must be identified for each topic. Each recommended reference item may, of course, cover multiple topics. Rarely, a topic may be self-descriptive and not cite a reference material item (e.g., a topic that is a definition or a topic for which the description itself without any cited reference material is sufficient for the objectives of the *SWEBOK Guide*).
  - » Each reference to the recommended reference material should be as precise as possible, identifying what specific chapter or section is relevant.
  - » A matrix of reference material (to the level of section number) versus topics must be provided.
  - » The latest versions or editions should be used as the Recommended References if there are multiple versions or editions.
  - » A reasonable amount of recommended reference material must be identified for each KA. The following guidelines should be used in determining how much is reasonable:
    i. If the recommended reference materials are written in a coherent manner, follow the proposed breakdown of topics, and use a consistent style (e.g., list a new book based on the proposed KA description), an average page number target across all KAs would be 750. However, this target may not be attainable when selecting existing reference material due to differences in style and to overlap and redundancy among the selected reference materials.
    i. In other words, the target for the number of pages for the entire collection of Recommended References in the *SWEBOK Guide* is in the range of 10,000 to 15,000 pages.
    i. Another way of viewing this is that the amount of recommended reference material would be reasonable if it consisted of the study material for this KA for a software engineering licensing exam that a graduate would pass after completing four years of work experience.

Specialized Practices Used Only for Certain Types of Software	
	Generally Recognized
	Established traditional practices recommended by many organizations
	Advanced and Research
	Innovative practices tested and used only by some organizations and concepts still being developed and tested in research organizations

Figure A.1. Categories of Knowledge

- Additional reference material can be included by the KA editor in a "Further Reading" list:
  - » These materials must be related to the topics in the breakdown rather than, for example, to more advanced topics.
  - » The list must be annotated (one paragraph per reference) to explain why each reference was included. Further Reading could include alternative viewpoints on a KA or a seminal treatment of a KA.
  - » A general guideline to be followed is 10 or fewer further readings per KA.
  - » There is no matrix of the reference materials listed in Further Reading and the breakdown of topics.
- Criteria and requirements regarding additional references cited in the KA Description:
  - » The *SWEBOK Guide* is not a research document, and its readership will be varied. Therefore, a delicate balance must be maintained between ensuring a high level of readability within the document and maintaining its technical excellence. Additional reference material should, therefore, be brought in by the KA editor only if it is necessary to the discussion. For example, the reference material might identify the source of a quotation or offer support for the rationale behind an important argument.

COMMON STRUCTURE

KA Descriptions should use the following structure:

- Acronyms
- Introduction
- Breakdown of Topics of the KA (including a figure describing the breakdown)
- Matrix of Topics vs. Reference Material
- Further Reading
- References

WHAT DO WE MEAN BY "GENERALLY RECOGNIZED KNOWLEDGE"?

The Software Engineering Body of Knowledge is an all-inclusive term that describes the sum of knowledge within the profession of software engineering. However, the *SWEBOK Guide* seeks to identify and describe the subset of the body of knowledge that is generally recognized or, in other words, the core body of knowledge. To better illustrate what "generally recognized" knowledge is relative to other types of knowledge, Figure A.1 proposes a three-category schema for classifying knowledge.

The Project Management Institute, in its *Guide to the Project Management Body of Knowledge*, defines "generally recognized" knowledge for project management as:

> *that subset of the project management body of knowledge generally recognized as good practice. "Generally recognized" means the knowledge and practices described are applicable to most projects most of the time, and there is consensus about their value and usefulness. "Good practice" means there is general agreement that the application of these skills, tools, and techniques can enhance the chances of success over a wide range of projects. "Good practice" does not mean that the knowledge described should always be applied uniformly to all projects; the organization and/or project management team is responsible for determining what is appropriate for any given project [1].*

"Generally accepted" knowledge could also be viewed as knowledge to be included in the study material of a software engineering licensing exam (in the US) that a graduate would take after completing four years of work experience. These two definitions should be seen as complementary.

KA editors are also expected to be somewhat forward-looking in their interpretation by taking into consideration not only what is "generally recognized" today but also what they expect will be "generally recognized" in a three- to five-year time frame.

LENGTH OF KA DESCRIPTION

KA Descriptions are to be roughly 10 to 20 pages using the formatting template for papers published in conference proceedings of the IEEE Computer Society. This includes text, references, appendixes, tables, etc. This, of course, does not include the reference materials themselves.

IMPORTANT RELATED DOCUMENTS

Graduate Software Engineering 2009 (GSwE2009): Curriculum Guidelines for Graduate Degree Programs in Software Engineering, 2009 [2].

This document "provides guidelines and recommendations" for defining the curricula of a professional master's-level program in software engineering. The *SWEBOK Guide* is identified as a "primary reference" in developing the body of knowledge underlying these guidelines. This document has been officially endorsed by the IEEE Computer Society and sponsored by the Association for Computing Machinery.

ISO/IEC/IEEE 12207:2017 Standard for Systems and Software Engineering — Software Life Cycle Processes [3].

This standard is considered the key standard regarding the definition of life cycle processes and has been adopted by the two main standardization bodies in software engineering: ISO/IEC JTC1/SC7 and the IEEE Computer Society Software and Systems Engineering Standards Committees. It also has been designated a pivotal standard by the Software and Systems Engineering Standards Committee (S2ESC) of the IEEE.

Even though we do not intend the *SWEBOK Guide* to be fully 12207-conformant, this standard remains a key input to the *SWEBOK Guide*, and special care will be taken throughout the *SWEBOK Guide* regarding the compatibility of the *Guide* with the 12207 standard.

"Software Engineering 2014: Curriculum Guidelines for Undergraduate Degree Programs in Software Engineering," IEEE Computer Society and Association for Computing Machinery, 2015; https://www.acm.org/binaries/content/assets/education/se2014.pdf [4].

This document describes curriculum guidelines for an undergraduate degree in software engineering. The *SWEBOK Guide* is identified as "one of the primary sources" in developing the body of knowledge underlying these guidelines.

ISO/IEC/IEEE 24765:2017 Software and Systems Engineering — Vocabulary; https://www.computer.org/sevocab [5].

The hierarchy of references for terminology is *Merriam-Webster's Collegiate Dictionary* (11th ed.) [6], ISO/IEC/IEEE 24765 [5] and newly proposed definitions, if required.

"Software Professional Certification Program," IEEE Computer Society; https://www.computer.org/education/certifications [7].

Information on the certification and associated professional development products developed and offered by the IEEE Computer Society for professionals in the field of software engineering can be found on this

website. The *SWEBOK Guide* is foundational to these products.

OTHER DETAILED GUIDELINES

When referencing the *Guide to the Software Engineering Body of Knowledge*, use the title *SWEBOK Guide*.

For the purpose of simplicity, avoid footnotes, and try to include their content in the main text.

Use explicit references to standards, as opposed to simply inserting numbers referencing items in the bibliography. We believe this approach allows the reader to be better exposed to the source and scope of a standard.

The text accompanying figures and tables should be self-explanatory or have enough related text. This ensures that the reader knows what the figures and tables mean.

To make sure that some information in the *SWEBOK Guide* does not become rapidly obsolete and in order to reflect its generic nature, please avoid directly naming tools and products. Instead, try to name their functions.

EDITING

Editors of the *SWEBOK Guide*, as well as professional copy editors, will edit KA Descriptions. Editing includes copy editing (grammar, punctuation and capitalization), style editing (conformance to the Computer Society style guide), and content editing (flow, meaning, clarity, directness and organization). The final editing will be a collaborative process in which the editors of the *SWEBOK Guide* and the KA editors will work together to achieve a concise, well-worded and useful KA Description.

RELEASE OF COPYRIGHT

All intellectual property rights associated with the *SWEBOK Guide* will remain with the IEEE. KA editors must sign a copyright release form.

It is also understood that the *SWEBOK Guide* will continue to be available free of charge in the public domain in at least one format, provided by the IEEE Computer Society through web technology or by other means.

(For more information, see www.computer.org/copyright.htm.)

REFERENCES

[1] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 7th ed., Project Management Institute, 2021.

[2] Integrated Software and Systems Engineering Curriculum (iSSEc) Project, Graduate Software Engineering 2009 (GSwE2009): Curriculum Guidelines for Graduate Degree Programs in Software Engineering, Stevens Institute of Technology, 2009; https://dl.acm.org/doi/book/10.1145/2593248.

[3] ISO/IEC/IEEE 12207:2017 Systems and Software Engineering — Software Life Cycle Processes.

[4] Joint Task Force on Computing Curricula, IEEE Computer Society and Association for Computing Machinery, "Software Engineering 2014: Curriculum Guidelines for Undergraduate Degree Programs in Software Engineering," 2015; https://www.acm.org/binaries/content/assets/education/se2014.pdf.

[5] ISO/IEC/IEEE 24765:2017 Systems and Software Engineering — Vocabulary, 2nd ed.

[6] *Merriam-Webster's Collegiate Dictionary*, 11th ed., 2003.

[7] IEEE Computer Society, "Certification and Training for Software Professionals," 2013; https://www.computer.org/education/certifications.

IEEE AND ISO/IEC STANDARDS

Appendix B

ACRONYMS

EIC	International Electrotechnical Commission
ISO	International Organization for Standardization
JTC	Joint Technical Committee
MSS	Management System Standard
S2ESC	Systems and Software Engineering Standards Committee
SC	Subcommittee

SUPPORTING THE SOFTWARE ENGINEERING BODY OF KNOWLEDGE (SWEBOK)

1. Overview

The purpose of this appendix is to describe the relationship between IEEE software engineering standards and the SWEBOK and to introduce the more prominent international software engineering standards most directly related to the SWEBOK knowledge areas (KA). A summary list of some useful standards for software engineering, including all those referenced in this document, is in B.9.

1.1 The SWEBOK and standards

The SWEBOK and other bodies of knowledge are closely related to standards for software engineering, and standards are cited as resources in knowledge areas (KA) in the SWEBOK. Standards for software engineering extend and apply the generally accepted body of knowledge that is collected in the SWEBOK. Conversely, standards also define and organize the systematic knowledge that is then reflected in collected bodies of knowledge. However, the SWEBOK has a different purpose from most software engineering standards. The SWEBOK summarizes generally accepted concepts and experience-based information about how software engineering is practiced. This knowledge summary can be applied in various ways: to define a curriculum for educating software engineers, or for employers or certification bodies determine if a person has the knowledge and accepts the ethical values needed to practice software engineering or to be certified.

In contrast, a standard is a "document, established by consensus and approved by a recognized body, that provides, for common and repeated use, rules, guidelines or characteristics for activities or their results, aimed at the achievement of the optimum degree of order in a given context" (ISO/IEC 29110-1-1:2024). In standards, the "Rules, guidelines, or characteristics" are expressed differently:

- requirements in normative standards, (stated using *shall* or the imperative),
- recommended practices (stated using *should*)
- other guidance on possible approaches (stated using *may)*

Standards allow for global interoperability for accepted concepts, processes, people, and products. The existence of standards takes a very large (possibly infinite) trade space of alternatives and normalizes that space, supporting mutual understanding between acquirers and suppliers. In that respect, software engineering standards counter the tendency of competing organizations to develop unique, proprietary

products that do not interoperate outside their own suite. When standards are open, so that organizations of all sizes can meet their requirements, demand for trustworthy products and services increases to the benefit of many suppliers and acquirers.

Standards are voluntary; an individual or organization can choose to conform to their requirements and follow their recommendations. When the standard is incorporated in contracts or other agreements, laws, and regulations, then compliance with the standard becomes mandatory.

1.2 Types of Standards

Standards can be characterized by what part of software engineering they standardize: concepts and terms, processes, products, people, or assessment of capabilities.

Some software engineering standards simply present concepts (characteristics) and define terms, perhaps even establishing a schema of knowledge about a software engineering topic. An example of this type of standard is ISO/IEC/IEEE 24765 *Systems and software engineering: Vocabulary*, which is freely available online at www.computer.org/sevocab.[1] However, most software engineering standards describe one or more of the software engineering processes and give requirements and recommendations about how to perform that process. The primary process standard in software engineering is ISO/IEC/IEEE 12207, *Systems and software engineering: Software life cycle processes*. There is even a standard for how to describe a process: ISO/IEC/IEEE 24774, *Systems and software engineering — Life cycle management — Specification for process description*. It describes the purpose, outcomes, activities, tasks, and possibly the inputs, outputs, and other features of a process. Process standards should not be confused with procedures or instructions; they do not offer detailed recipes or step-by-step instructions for doing software engineering.

A few software engineering standards have standardized descriptions of products of software engineering, such as models or information products like a project management plan (ISO/IEC/IEEE 16326). Another notable standard for information products is ISO/IEC/IEEE 15289, *Systems and software engineering — Contents of life cycle information items (documentation)*. Initially, most software engineering standards were standards for a prominent information product, a plan. These allowed customers (acquirers of software) to understand and compare what their suppliers would produce (a product). A standard for a plan describes what will be produced or delivered, what methods and techniques will be used, and what activities will be performed. In recent years, most of the standards for plans have been revised to become standards for software engineering processes.

Besides standards for concepts, processes, and products, there are also standards for people's skills, knowledge, or abilities, and standards for certification schemes and bodies of knowledge in software engineering. An example is ISO/IEC 24773-1:2019, *Software and systems engineering — Certification of software and systems engineering professionals — Part 1: General requirements*. Reviews and assessments can be standardized for software engineers, organizations, processes, and work products.

1.3 Sources of Software Engineering Standards

Although there are thousands of pages in hundreds of systems and software engineering standards, guides, textbooks and handbooks, there are only two international organizations accredited to produce systems and software engineering standards: ISO/IEC JTC 1/SC 7 and IEEE. Both have produced software engineering standards for over thirty-five years. Both are committed to produce standards using documented, consensus-based processes with open participation. ISO/IEC JTC 1 (International Organization for Standardization / International

1 http://pascal.computer.org/sev_display/index.action.

Electrotechnical Commission Joint Technical Committee) / SC 7 (Subcommittee), Software and Systems Engineering, produces standards through its membership of national standards bodies. JTC 1/SC 7 has a portfolio of over two hundred standards. The IEEE Computer Society Systems and Software Standards Committee (S2ESC) produces standards in working groups of individual experts. It maintains about fifty standards, of which about 80% have been approved as ISO/IEC/IEEE joint standards. These are IEEE standards adopted by ISO/IEC JTC 1/SC 7, or standards that are jointly developed and maintained with ISO/IEC JTC 1 and designated as ISO/IEC/IEEE. The aim of these jointly developed standards is to have a coordinated collection of consistent standards for international use. For the ISO/IEC/IEEE standards described in this appendix, the IEEE version and the ISO/IEC version are substantively identical. The respective versions may have different front and back matter but the technical content is exactly the same.

Standards can be purchased from the IEEE, ISO, and IEC websites, from national standards organizations, and from commercial resellers. Academic institutions and software engineering organizations can purchase or subscribe to collections of standards for use by their staffs. A few standards are freely available, generally those that provide introductions to concepts or terminology.

In both IEEE and ISO/IEC JTC 1, standards for systems engineering are maintained by the same committee as those for software engineering. Most of the standards apply to both, especially when software is considered as a system or as the major component of a system of interest. So, instead of making fine distinctions, this appendix covers both as applicable to software engineering. It does not mention older, now stabilized standards dealing with the foundations of computing or computing languages and basic programming, mathematical, or engineering concepts.

ISO and IEEE have their own numbering systems for their standards. When an IEEE standard is adopted by ISO/IEC JTC 1, it is typically renumbered to a 5-digit number, e.g., IEEE 1062 becomes ISO/IEC/IEEE 41062. ISO standards have long, taxonomical titles with three and four levels of classification. The first level shows the general area (e.g. systems and software engineering); the second level is the main title of the standard, and the third level provides even more detail, especially for multi-part standards. To avoid cumbersome repetition, this appendix often uses a shortened title of the standard or simply cites it by number. The full title is given in the list in B.9. All of these software engineering standards are copyright protected, and IEEE standard numbers are trademarked.

2. The software engineering standards landscape

Figure B.1 presents an overview of the most prominent software engineering standards, mainly from the perspective of how other standards relate to the major software engineering life cycle process standard, ISO/IEC/IEEE 12207, software engineering processes. It is closely related to the SWEBOK in that both present information related to many of the same software life cycle processes. Also in the upper portion of Figure B.1 are the foundational standards, such as the specialized vocabulary for systems and software engineering (SEVOCAB, ISO/IEC/IEEE 24765) and a specification for how to describe processes (ISO/IEC/IEEE 24774). There are standards for how to plan for and manage software engineering (ISO/IEC/IEEE 24748-5) and how to conduct rigorous reviews and audits, appropriate for critical software like aerospace and defense systems (ISO/IEC/IEEE 24748-8).

Using the life cycle process model of 12207 as described in the following section, there are many more specialized standards covering individual processes and modern approaches to the processes, such as ISO/IEC/IEEE 32675, DevOps, as well as IEEE 1012, Verification and validation, and ISO/IEC/IEEE 29119, software testing (in multiple parts). The life cycle processes in 12207 generally focus on a

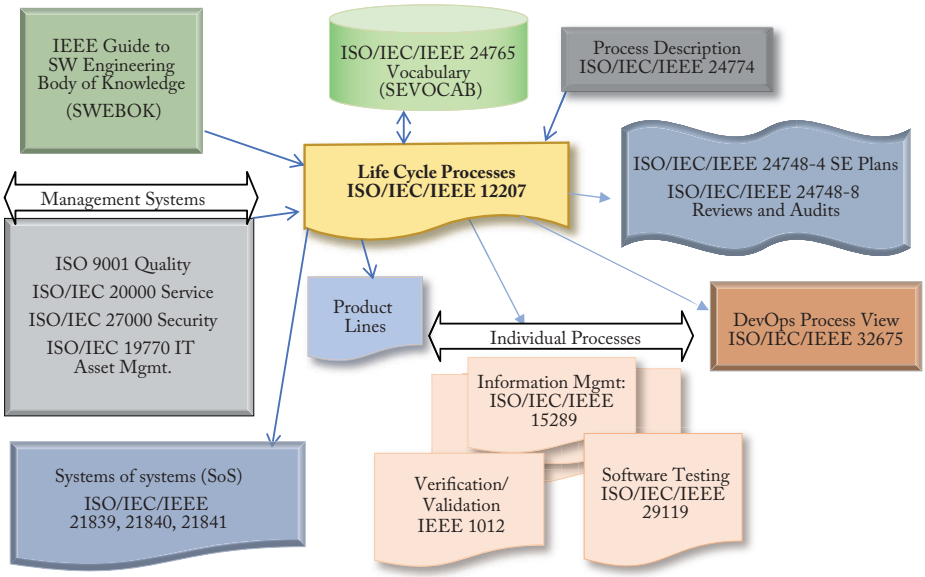

Figure B.1. Software Engineering Standards Landscape

single system of interest (SOI) but more specialized series focus on processes and tools for product line engineering, and for systems of systems (SoS). The System of Systems standards, ISO/IEC/IEEE 21839, 21840, and 21841, explain how to use systems engineering processes when the system of interest (SOI) is a constituent part of a system of systems.

The life cycle process standards are intended to be compatible with other well-known standards for management systems. According to ISO, "a management system is the way in which an organization manages the interrelated parts of its business in order to achieve its objectives." Management system standards (MSS) have a consistent structure and framework of requirements, but each MSS covers a specific aspect of managing and delivering engineering products and services. MSS typically come in multiple parts with various guides for different aspects of their systems. Well-known MSS related to software engineering include ISO 9000 for quality management, ISO/IEC 20000 for service management, the ISO/IEC 27000 series for information security management, the ISO/IEC 19770 series for managing IT assets like hardware and software, and the ISO/IEC 30105 series for business process outsourcing operations.

3. Life cycle process standards

ISO/IEC/IEEE 12207, Software life cycle processes, and ISO/IEC/IEEE 15288, System life cycle processes, are intentionally harmonized for use together. As stated in ISO/IEC/IEEE 15288:2023, "there is a continuum of human-made systems from those that use little or no software to those in which software is the primary interest. When software is the predominant system or element of interest, ISO/IEC/IEEE 12207 should be used." Both standards have identical life cycle models (the same four process groups, as shown in Figure B.2) and the same processes. The processes have the same names, purposes, and process outcomes (there are minor variations in a couple of process names) in both standards. Process activities and tasks differ between these two foundational standards, as some aspects of engineering for software systems are different from systems in general. Conformance to ISO/IEC/IEEE 12207 or

15288 can be shown either by demonstrating that all the outcomes of the process have been achieved, or that all the required activities and tasks of a process have been performed.

The life cycle processes are presented in the context of their use on projects, supported by an organization that provides continuous services applicable across multiple projects. However, the processes can be applied in very small entities which are essentially organized as a single team, as well as on large programs and continuing efforts that do not have a defined end point like a project.

ISO/IEC/IEEE 12207 establishes a common framework for software life cycle processes, with well-defined terminology that can be referenced by the software industry. ISO/IEC 12207 applies to the acquisition of systems and software products and services and to the supply, development, operation, maintenance, and disposal of software systems and the software portion of a system, whether performed internally or externally to an organization. Those aspects of system definition and enabling systems (infrastructure) needed to provide the context for software products and services are included. Selected sets of these processes can be applied throughout the life cycle for managing and performing the stages of a system's life cycle. This is accomplished through the involvement of all interested parties, with the goal of achieving customer satisfaction.

Table B.1 aligns the software life cycle processes of ISO/IEC/IEEE 12207 to the SWEBOK KA and identifies related standards that offer more detailed requirements and guidance for individual processes. The SWEBOK KAs do not directly cover all of the process groups and processes in ISO/IEC/IEEE 12207. The Agreement processes (acquisition and supply) are not included, nor many of the processes in the Organizational Project-enabling process group, and not all of the Technical Management or Technical process group processes. SWEBOK KAs are selected to cover the essential knowledge areas applied by individual software engineers working on projects or ongoing efforts, rather than those generally handled at higher levels in the organization or on a more general level.

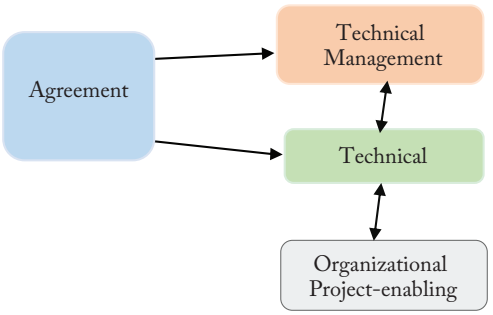

Figure B.2. Process groups of ISO/IEC/IEEE 12207

This version of the SWEBOK has added the software security KA, which for historical reasons has been standardized separately from the systems and software engineering standards committees. Security is not identified as a technical process in ISO/IEC/IEEE 12207. An extensive suite of security standards based on the ISO/IEC 27001 MSS are developed in ISO/IEC JTC 1 SC 27, Information security, cybersecurity, and privacy protection.

Table B.1 also identifies standards that are intended to identify process-related functions where software tools and methods should be applied, or to apply the processes to product line engineering (see B.6).

4. Extensions and specialized applications of ISO/IEC/IEEE 12207

Numerous useful standards supplement the requirements of ISO/IEC/IEEE 12207 to handle more rigorous or specialized situations, or to provide more extended guidance on its concepts and processes. Many of these standards are parts of the ISO/IEC/IEEE 24748 family.

4.1, Explanations of concepts and several processes

ISO/IEC/IEEE 24748-1, -2 and -3 are overall guides to the life cycle processes and invaluable for understanding and applying

TABLE B.1. RELATED SOFTWARE ENGINEERING STANDARDS AND KNOWLEDGE AREAS BY ISO/IEC/IEEE 12207 PROCESS GROUP AND PROCESS

12207 Clause Number	**Short title**	**SWEBOK KA**	**Related standard** (ISO/IEC/ IEEE unless otherwise shown)	**Product line or tool standard** (ISO/IEC)
Agreement				
6.1.1	Acquisition		41062, 26512	
6,1,2	Supply		41062	
Organizational process enabling				
6.2.1	Life cycle model management	Yes	24748-1, 24748-2, 24748-3, 33020	
6.2.2	Infrastructure management			26550
6.2.3	Portfolio management		33001	26556
6.2.4	Human Resources management	Yes, Professional Practice	24773	
6.2.5	Quality management	Yes, Software Quality	IEEE 730, 25000, 90003	
6.2.6	Knowledge management			
Technical management				
6.3.1	Project planning	Yes	16326, 24748-4, 24748-5	26555
6.3.2	Project assessment, control	Yes	16326, 24748-4, 24748-5, 24748-7, 26511, 20246	23396, 23531, 26555, 33001, 33002
6.3.3	Decision management			
6.3.4	Risk Management		16085, 15026 (all parts)	
6.3.5	Configuration Management	Yes	IEEE 828, 16350, 19770 (all parts)	26559, 26560, 26561
6.3.6	Information management		15289, 26511, 26531, 23026, 82079-1	
6.3.7	Measurement	Yes	15939, 14143, 32430, 19761, 20926, 25020, 25021, 25022, 25023, 25024, 29881, 33003	
6.3.8	Quality Assurance	Yes	IEEE 730, IEEE 982.1, 25010, 25012	

Technical				
6.4.1	Business or Mission Analysis			26561
6.4.2	Stakeholder needs & requirements	Yes	25030	
6.4.3	Systems requirements definition	Yes	29148	26551
6.4.4	Architecture definition	Yes	42010, 42020	26442, 26552
6.4.5	Design definition	Yes	24748-7000. 26514	26557, 26580
6.4.6	System analysis	Yes, Models and Methods	ISO/IEC 24641	20246, 26558
6.4.7	Implementation	Yes, Construction		26553
6.4.8	Integration	Yes, Construction	24748-6	
6.4.9	Verification	Yes, Testing	IEEE 1012, 25021,25040, 25041, 25045, 25062, 26513, 29119-1, 29119-2, 29119-3, 33063, 42030	23643,26554, 30130
6.4.10	Transition			26562
6.4.11	Validation	Yes, Testing	IEEE 1012	
6.4.12	Operation	Yes	32675	23531
6.4.13	Maintenance	Yes	14764	
6.4.14	Disposal			
	Software security	Yes	ISO/IEC 27000 family, 15026 (Parts 1 to 4)	
	Software Engineering computing foundations	Yes	Numerous historic standards	
	Software Engineering Mathematical foundations	Yes	Numerous historic standards	
	Software engineering Foundations	Yes	Numerous historic standards	

systems and software engineering concepts. ISO/IEC/IEEE 24748-1, Guidelines for life cycle management, is much more than a guide to performing the life cycle management process. It applies to both software and systems engineering processes, with further explanations of system and process concepts. Instead of describing processes, which are usually applied repeatedly throughout the life cycle, it includes a detailed description of life cycle stages, covering their purpose and outcomes. There are several models of life cycle stages, and in ISO/IEC/IEEE 24748-1 the model that is analyzed in detail includes the following stages: concept, development, production, utilization, support, and

retirement. Since software engineers rarely focus on production as a stage of interest, an alternate model for software life cycle stages is more useful: concept, development, operations and maintenance, and retirement. Life cycle models are characterized by their development approach: sequential, incremental, or evolutionary. The life cycle models are compared in a risk-based approach.

ISO/IEC/IEEE 24748-2 is the overall guide to applying the systems engineering processes in ISO/IEC/IEEE 15288. However, it does not offer line-by-line expansions of each process, activity, and task, but presents an overall strategy for transitioning to use of standardized life cycle processes. There is yet more explanation of systems concepts, a presentation of organizational concepts, some discussion of conformance or adaptation (tailoring), of standard processes, and an introduction to model-based systems and software engineering (MBSSE).

ISO/IEC/IEEE 24748-3, guidelines for the application of software life cycle processes, also offers commentary on concepts of software systems, organizations and projects, processes, life cycle states, and life cycle process models for software systems. It includes guidance for each of the processes in ISO/IEC/IEEE 12207, including further analysis of process purposes; outcomes and outputs; activities, tasks, and approaches; closely related processes; and related standards.

ISO/IEC/IEEE 32675 DevOps, (IEEE 2675) has the informative subtitle of "Building Reliable and Secure Systems, Including Application Build, Package, and Deployment". It defines DevOps as a "set of principles and practices which enable better communication and collaboration between relevant stakeholders for the purpose of specifying, developing, and operating software and systems products and services, and continuous improvements in all aspects of the life cycle." It expounds on the principles of DevOps, including business or mission first, customer focus, left shift and continuous everything, and systems thinking. (Left-shift is defined as "prioritizing the involvement of relevant stakeholders in applying quality activities, security, privacy, performance, verification, and validation earlier in the life cycle.") IEEE 2675 emphasizes the leadership commitment needed for successful application of DevOps. It reviews many of the life cycle processes in ISO/IEC/IEEE 12207 to analyze how they are transformed by DevOps, and discusses the use of DevOps with agile methods.

In earlier versions, both ISO/IEC/IEEE 24748-4 and 24748-5 covered what to include in a management plan (Systems Engineering Management Plan or Software Engineering Management Plan), respectively. That material is still there, but now they also include guidance for systems engineers and software engineers, respectively, on the management planning and control processes, with brief presentations of related processes.

4.2 More specialized extensions

Although standards are well established for specialized areas of health and safety, security, and environmental concerns, standards relating ethical values to software systems are relatively new. The potential for software systems to cause harm through biased decisions, violations of privacy, or lack of social responsibility led to the development of ISO/IEC/IEEE 24748-7000 (IEEE 7000). IEEE 7000 presents a model process for incorporating ethical values into systems design. Engineers, their managers, and other stakeholders benefit from well-defined processes for considering ethical issues along with the usual concerns of system performance and functionality early in the system life cycle. The standard requires consideration of values relevant to the culture where the system is to be deployed. It is applicable with any life cycle model or development methodology. The processes in this standard are intended to be performed concurrently with those in ISO/IEC/IEEE 12207 (Table B.3).

Earlier versions of ISO/IEC/IEEE 12207 were considered by some to be overly

prescriptive in terms of required documentation, reviews, and task sequences. The current version is intended to be used by any size or type of organization, having a more strategic, agile, approach to the processes, with reduced documentation and review requirements. However, for highly complex and critical systems, a more rigorous and structured set of processes, reviews, and audits was developed in coordination with the US Department of Defense and has been specified in ISO/IEC/IEEE 24748-7:2019 *Systems and software engineering — Life cycle management — Part 7: Application of systems engineering on defense programs* and ISO/IEC/IEEE 24748-8:2019 *Systems and software engineering — Life cycle management — Part 8: Technical reviews and audits on defense programs.* For more general use, ISO/IEC 20246 outlines processes and characteristics for work product reviews throughout the life cycle, covering both software and information products.

ISO/IEC/IEEE 24748-9 is an application of system and software life cycle processes in epidemic prevention and control systems. More generally, it shows ways of doing systems and software engineering with limited infrastructure and staff support, such as "insufficient infrastructure protection, short delivery cycles, frequent iterative upgrades, and special requirements such as accuracy, disaster tolerance, degradation capability, safety, user capacity and stress testing, and rapid demand capture."

4.3 SoS standards

Three standards explore how the systems and software engineering concepts and processes can be applied to systems of systems (SoS). ISO/IEC/IEEE 21839 describes how systems that are constituents of SoS are affected at each stage in their life cycle. ISO/IEC/IEEE 21940 takes the opposite view, exploring concepts of an SoS and how ISO/IEC/IEEE 15288 can be applied to SoS. ISO/IEC/IEEE 21841 is a brief taxonomy that identifies four types of SoS: directed, acknowledged, collaborative and virtual.

5. Single Process Standards

ISO/IEC/IEEE 12207 applies to all types of software engineering with a variety of life cycle models, techniques, and methods. Its process descriptions do not go into detail about how the process should be performed or which techniques are considered best practice. To that end, there are numerous more specialized standards with additional requirements and guidelines applicable to most of the software engineering processes. Table B.1 correlates each process in ISO/IEC/IEEE 12207 to the related SWEBOK KAs, more specialized standards and guidance, and related standards for applying the process to product lines, tools, and methods. Table B.2 shows standards referenced in each knowledge area.

6. Standards for product line, methods, and tools

A product line is a "set of products or services sharing explicitly defined and managed common and variable features and relying on the same domain architecture to meet the common and variable needs of specific markets" (ISO/IEC 26550:2015) Product line engineering raises different considerations, especially for ongoing configuration and release management, maintenance, and operations, from the basic approach of ISO/IEC/IEEE 12207, which applies software engineering from the perspective of a project within an organization.

Standards in the ISO/IEC 26550 to 26569 series also cover capabilities of tools related to various software engineering processes and management tasks. Because software development and operations tool capabilities are continually being expanded and more tightly integrated to support the DevOps pipeline, the individual standards in this series are not closely aligned with current commercial product suites or open-source libraries. However, the tool standards do suggest useful features to seek in support of the software lifecycle.

TABLE B.2. STANDARDS CITED BY KNOWLEDGE AREA

KA Number	Knowledge Area	Cited standards (ISO/IEC/IEEE unless otherwise designated)
	Introduction	24765, 12207
1	Software Requirements	24765, 12207, ISO/IEC 25010, 29148
2	Software Architecture	24765, 12207, 42010
3	Software Design	12207, 24748-7000, 24765
4	Software Construction	
5	Software Testing	IEEE 1012, ISO/IEC 20246, 24765, ISO/IEC 25010, 29119 (multiple parts), 32675
6	Software Engineering Operations	12207, ISO/IEC 20000, 24765, 32675
7	Software Maintenance	12207, 14764, 15288, 32675
8	Software Configuration Management	IEEE 828, 24765, 12207
9	Software Engineering Management	12207, 32675
10	Software Engineering Process	12207, 24748-1, 24748-3, 24765, 24774, ISO/IEC 25000, 29110, 33001, 32675
11	Software Engineering Models and Methods	
12	Software Quality	IEEE 730, IEEE 982.1, IEEE 1012, IEEE 1228, IEEE 1633, ISO 9001, 12207, 15026-1, 15288, 20000, 20246, 24765, 25010, 27001, 33061, 90003, IEC 60300
13	Software Security	ISO/IEC 15408-1, ISO/IEC 18045, ISO/IEC 19770-1, ISO/IEC 21827, 25010, ISO/IEC 27000, ISO/IEC 27001, ISO/IEC 27032
14	Software Engineering Professional Practice	ISO/IEC 24773-1, ISO/IEC 24773-4
15	Software Engineering Economics	12207, 15288
16	Computing Foundations	12207, 24765
17	Mathematical Foundations	
18	Engineering Foundations	24765

7. Process assessment standards

Process assessment is a long-standing method of confirming the capabilities, quality, and maturity of software engineering processes, and encouraging process improvement. Process audits look for evidence of performance of activities and achievement of outcomes (artifacts like work products and information items). The assumption is that a repeatable process with organizational support performed by competent practitioners is more likely to produce acceptable software products and services. The ISO/IEC 33000 family of standards

TABLE B.3. ALIGNMENT OF ETHICAL VALUE PROCESSES IN ISO/IEC/IEEE 24748-7000 (IEEE 7000) AND SOFTWARE ENGINEERING PROCESSES IN ISO/IEC/IEEE 12207

IEEE Std 7000 process clause	ISO/IEC/IEEE 12207:2017 and ISO/IEC/IEEE 15288:2023 process clause
7. Concept of Operations (ConOps) and Context Exploration	*6.4.1 Business or mission analysis*
8. Ethical Values Elicitation and Prioritization	*6.4.1 Business or mission analysis,* *6.4.2 Stakeholder needs and requirements definition*
9. Ethical Requirements Definition	*6.4.2 Stakeholder needs and requirements definition,* *6.4.3 System requirements definition*
10. Ethical Risk-Based Design	*6.4.4 Architecture definition,* *6.4.5 Design definition*
11. Transparency Management	*6.3.6 Information management*

currently includes over twenty active standards related to process assessment. The overall architecture and content of the ISO/IEC 330xx family is described in ISO/IEC 33001. A process assessment is conducted according to a documented assessment process, which identifies the rating method for process attributes and how to determine process ratings. ISO/IEC 33061 is the standard for process assessment which is aligned with ISO/IEC/IEEE 12207 software engineering processes, treated as a process reference model.

8. Professional Skills and Knowledge Standards

The ISO/IEC 24773 series contains requirements specifically related to certifications for software and systems engineering professionals. It is useful to industry organizations seeking to compare various certifications for professionals in systems and/or software engineering; to individual professionals seeking to obtain certification; and to employers who may choose to recognize such certifications. These standards are intended for international use, and do not replace national or regional licensing or registration requirements for engineers. ISO/IEC 24773-1 is an overview of certification concepts, and requirements for the certification processes and certification schemes applicable to software and systems engineering. ISO/IEC 24773-4 provides specific requirements for certification bodies in software engineering. It specifies this IEEE SWEBOK as the reference body of knowledge in software engineering.

9. Selected Software Engineering Standards

This is not an exhaustive list of standards related to software engineering or sponsored by the IEEE Systems and Software Engineering Standards Committee (S2ESC) or ISO/IEC JTC 1/SC7, Software and systems engineering. Those listed are considered more authoritative, relevant, and helpful for SWEBOK users.

The standards described in this appendix are continually being revised or replaced by newer standards. Users of standards should look for the most recent version and for newer titles relating to emerging topics in software engineering, such as digital engineering or standards related to artificial intelligence (AI).

- IEEE 730-2014 IEEE Standard for Software Quality Assurance Processes
- IEEE 828-2012 IEEE Standard for Configuration Management in Systems and Software Engineering

- IEEE 982.1-2005 IEEE Standard Dictionary of Measures of the Software Aspects of Dependability
- IEEE 1012-2016 IEEE Standard for System, Software, and Hardware Verification and Validation
- IEEE 1228-1994 (R2002) IEEE Standard for Software Safety Plans
- IEEE 1633-2016 IEEE Recommended Practice on Software Reliability
- ISO 9000:2015 Quality management systems — Fundamentals and vocabulary
- ISO 9001:2015 Quality management systems — Requirements
- ISO/IEC/IEEE 12207:2017 Systems and software engineering: Software engineering processes
- ISO/IEC 14143 Information technology — Software measurement — Functional size measurement (multiple parts)
- ISO/IEC/IEEE 14764:2021 Software Engineering — Software Life Cycle Processes - Maintenance
- ISO/IEC/IEEE 15026-1:2019 Systems and Software Engineering — Systems and Software Assurance — Part 1: Concepts and Vocabulary
- ISO/IEC/IEEE 15026-2:2021 Systems and Software Engineering — Systems and Software Assurance — Part 2: Assurance Case
- ISO/IEC 15026-3:2023 Systems and Software Engineering — Systems and Software Assurance — Part 3: System Integrity Levels
- ISO/IEC/IEEE 15026-4:2021, Systems and Software Engineering — Systems and Software Assurance — Part 4: Assurance in the Life Cycle
- ISO/IEC/IEEE 15288:2023 Standard for Systems and Software Engineering — System Life Cycle Processes
- ISO/IEC/IEEE 15289:2019 Systems and Software Engineering — Content of Life-Cycle Information Products (Documentation)
- ISO/IEC 15408-1:2022 Information security, cybersecurity and privacy protection — Evaluation criteria for IT security — Part 1: Introduction and general model
- ISO/IEC/IEEE 15939:2017 Systems and Software Engineering — Measurement Process
- ISO/IEC/IEEE 16085:2021 Systems and Software Engineering —Software Life Cycle Processes — Risk Management
- ISO/IEC/IEEE 16326:2019 Systems and Software Engineering — Life Cycle Processes — Project Management
- ISO/IEC 16350:2015 Information technology — Systems and software engineering — Application management
- ISO/IEC 18045:2022 Information security, cybersecurity and privacy protection — Evaluation criteria for IT security — Methodology for IT security evaluation
- ISO/IEC 19761:2011 Software Engineering — COSMIC: A Functional Size Measurement Method
- ISO/IEC 19770-1:2017 Information technology — IT asset management — Part 1: IT asset management systems — Requirements
- ISO/IEC 19770-2:2015 Information technology — IT asset management — Part 2: Software identification tag
- ISO/IEC 19770-3:2016 Information technology — IT asset management — Part 3: Entitlement schema
- ISO/IEC 19770-4:2017 Information technology — IT asset management — Part 4: Resource utilization measurement
- ISO/IEC 19770-5:2015 Information technology — IT asset management — Part 5: Overview and vocabulary
- ISO/IEC 19770-8:2020 Information technology — IT asset management — Part 8: Guidelines for mapping of industry practices to/from the ISO/IEC 19770 family of standards
- ISO/IEC 19770-11:2021 Information technology — IT asset management — Part 11: Requirements for bodies providing audit and certification of IT asset management systems
- ISO/IEC 20000-1:2018 Information Technology — Service Management

— Part 1: Service management system requirements
- ISO/IEC 20246:2017 Software and systems engineering — Work product reviews
- ISO/IEC 20741:2017 Systems and software engineering — Guideline for the evaluation and selection of software engineering tools
- ISO/IEC 20926:2009 Software and Systems Engineering — Software Measurement — IFPUG Functional Size Measurement Method
- ISO/IEC 20968:2002 Software Engineering — Mk II Function Point Analysis — Counting Practices Manual
- ISO/IEC 21827:2008 Information technology — Security techniques — systems security engineering — capability maturity model® (SSE-CMM®)
- ISO/IEC/IEEE 21839:2019 Systems and software engineering — system of systems (SoS) considerations in life cycle stages of a system
- ISO/IEC/IEEE 21840:2019 Systems and software engineering — Guidelines for the utilization of ISO/IEC/IEEE 15288 in the context of system of systems (SoS)
- ISO/IEC/IEEE 21841:2019 Systems and software engineering — Taxonomy of systems of systems
- ISO/IEC/IEEE 23026:2023 Systems and software engineering — Engineering and management of websites for systems, software, and services information
- ISO/IEC 23396:2020 Systems and software engineering — Capabilities of review tools
- ISO/IEC 23531:2020 Systems and software engineering — Capabilities of issue management tools
- ISO/IEC 24570:2018 Software engineering — NESMA functional size measurement method — Definitions and counting guidelines for the application of function point analysis
- ISO/IEC/IEEE 24641:2023 Systems and Software engineering — Methods and tools for model-based systems and software engineering
- ISO/IEC/IEEE 24748-1:2024 Systems and software engineering — Life cycle management — Part 1: Guidelines for life cycle management
- ISO/IEC/IEEE 24748-2:2024 Systems and software engineering — Life cycle management — Part 2: Guidelines for the application of ISO/IEC/IEEE 15288 (system life cycle processes)
- ISO/IEC/IEEE 24748-3:2020 Systems and software engineering — Life cycle management — Part 3: Guidelines for the application of ISO/IEC/IEEE 12207 (software life cycle processes)
- ISO/IEC/IEEE 24748-4:2016 Systems and software engineering — Life cycle management — Part 4: Systems engineering planning
- ISO/IEC/IEEE 24748-5:2017 Systems and software engineering — Life cycle management — Part 5: Software development planning
- ISO/IEC/IEEE 24748-6:2023, Systems and Software Engineering — Life Cycle Management — Part 6: Systems and Software Integration
- ISO/IEC/IEEE 24748-7:2019 Systems and software engineering — Life cycle management — Part 7: Application of systems engineering on defense programs
- ISO/IEC/IEEE 24748-8:2019 Systems and software engineering — Life cycle management — Part 8: Technical reviews and audits on defense programs
- ISO/IEC/IEEE 24748-9:2023 Systems and software engineering, prevention and control systems
- ISO/IEC/IEEE 24748-7000:2022 Model Process for Addressing Ethical Concerns during System Design
- ISO/IEC/IEEE 24765:2017 Systems and Software Engineering — Vocabulary, available at www.computer.org/sevocab
- ISO/IEC 24773-1:2019 Software and systems engineering — Certification of software and systems engineering professionals — Part 1: General requirements
- ISO/IEC 24773-4:2023 Software and

systems engineering — Certification of software and systems engineering professionals — Part 4: Software engineering
- ISO/IEC/IEEE 24774:2021 Systems and software engineering — Life cycle management — Specification for process description
- ISO/IEC 25000:2014 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Guide to SQuaRE
- ISO/IEC 25001:2014 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — planning and management
- ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models
- ISO/IEC 25012:2008 Software engineering — Software product Quality Requirements and Evaluation (SQuaRE) — Data quality model
- ISO/IEC 25020:2019 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Quality measurement framework
- ISO/IEC 25021:2012 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Quality measure elements
- ISO/IEC 25022:2016 Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — Measurement of quality in use
- ISO/IEC 25023:2016 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Measurement of system and software product quality
- ISO/IEC 25024:2015 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Measurement of data quality
- ISO/IEC 25030:2019 Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — Quality requirements framework
- ISO/IEC 25040:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Evaluation process
- ISO/IEC 25041:2012 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Evaluation guide for developers, acquirers and independent evaluators
- ISO/IEC 25045:2010 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Evaluation module for recoverability
- ISO/IEC 25051:2014 Software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Requirements for quality of Ready to Use Software Product (RUSP) and instructions for testing
- ISO/IEC 25062 Software Product Quality Requirements and Evaluation (SQuaRE) — Common Industry Format (CIF) for Usability
- ISO/IEC 26442:2019 Software and systems engineering — Tools and methods for product line architecture design
- ISO/IEC/IEEE 26511:2018 Systems and software engineering — Requirements for managers of information for users of systems, software, and services
- ISO/IEC/IEEE 26512:2018 Systems and software engineering — Requirements for acquirers and suppliers of information for users
- ISO/IEC/IEEE 26513:2017 Systems and software engineering — Requirements for testers and reviewers of information for users
- ISO/IEC 26514:2021 Systems and Software Engineering — Design and development of information for users
- ISO/IEC/IEEE 26515:2018 Systems and

software engineering — Developing information for users in an agile environment
- ISO/IEC/IEEE 26531:2023 Systems and software engineering — Content management for product life-cycle, user and service management documentation
- ISO/IEC 26550:2015 Software and systems engineering — Reference model for product line engineering and management
- ISO/IEC 26551:2016 Software and systems engineering — Tools and methods for product line requirements engineering
- ISO/IEC 26552:2019 Software and systems engineering — Tools and methods for product line architecture design
- ISO/IEC 26553:2018 Information technology — Software and systems engineering — Tools and methods for product line realization
- ISO/IEC 26554:2018 Information technology — Software and systems engineering — Tools and methods for product line testing
- ISO/IEC 26555:2015 Software and systems engineering — Tools and methods for product line technical management
- ISO/IEC 26556:2018 Information technology — Software and systems engineering — Tools and methods for product line organizational management
- ISO/IEC 26557:2016 Software and systems engineering — Methods and tools for variability mechanisms in software and systems product line
- ISO/IEC 26558:2017 Software and systems engineering — Methods and tools for variability modelling in software and systems product line
- ISO/IEC 26559:2017 Software and systems engineering — Methods and tools for variability traceability in software and systems product line
- ISO/IEC 26560:2019 Software and systems engineering — Tools and methods for product line product management
- ISO/IEC 26561:2019 Software and systems engineering — Methods and tools for product line technical probe
- ISO/IEC 26562:2019 Software and systems engineering — Methods and tools for product line transition management
- ISO/IEC 26580:2021 Software and systems engineering — Methods and tools for the feature-based approach to software and systems product line engineering
- ISO/IEC 27000:2018 Information technology — Security techniques — Information security management systems — Overview and vocabulary
- ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection — Information security management systems — Requirements
- ISO/IEC 27032:2012 Information technology — Security techniques — Guidelines for cybersecurity
- ISO/IEC 29110-1-1:2024 Systems and software engineering - Lifecycle profiles for very small entities (VSEs) Part 1-1: Overview
- ISO/IEC 29110-2-1:2015 Software engineering — Lifecycle profiles for Very Small Entities (VSEs) — Part 2-1: Framework and taxonomy
- ISO/IEC TR 29110-5-3:2018 Systems and software engineering — Lifecycle profiles for Very Small Entities (VSEs) — Part 5-3: Service delivery guidelines
- ISO/IEC/IEEE 29119-1: 2022 Software and systems engineering — Software testing — Part 1: Concepts and definitions
- ISO/IEC/IEEE 29119-2: 2021 Software and systems engineering — Software testing — Part 2: Test processes
- ISO/IEC/IEEE 29119-3: 2021 Software and systems engineering — Software testing — Part 3: Test documentation
- ISO/IEC/IEEE 29119-4 Software and systems engineering — Software testing — Part 4: Test techniques
- ISO/IEC/IEEE 29119-5: 2016 Software and systems engineering — Software testing — Part 5: Keyword-Driven Testing
- ISO/IEC TR 29119-6:2021 Software and systems engineering — Software testing — Part 6: Guidelines for the use

of ISO/IEC/IEEE 29119 (all parts) in agile projects
- ISO/IEC TR 29119-11:2020 Software and systems engineering — Software testing — Part 11: Guidelines on the testing of AI-based systems
- ISO/IEC/IEEE 29148:2018. Systems and Software Engineering — Life Cycle Processes — Requirements Engineering
- ISO/IEC 30130:2016 Software engineering — Capabilities of software testing tools
- ISO/IEC 33001:2015 Information technology — Process assessment — Concepts and terminology
- ISO/IEC 33002:2015 Information technology — Process assessment — Requirements for performing process assessment
- ISO/IEC 33003:2015 Information technology — Process assessment — Requirements for process measurement frameworks
- ISO/IEC 33004:2015 Information technology — Process assessment — Requirements for process reference, process assessment and maturity models
- ISO/IEC TR 33014:2013 Information technology — Process assessment — Guide for process improvement
- ISO/IEC 33020:2019 Information technology — Process assessment — Process measurement framework for assessment of process capability
- ISO/IEC TS 33061:2021 Information technology — Process assessment — Process assessment model for software life cycle processes
- ISO/IEC 33063:2015 Information technology — Process assessment — Process assessment model for software testing
- ISO/IEC/IEEE 32430 Software engineering — Standard for software non-functional size measurements
- ISO/IEC/IEEE 32675:2021 DevOps: Building Reliable and Secure Systems Including Application Build, Package, and Deployment
- ISO/IEC 38500:2008 Corporate governance of information technology
- ISO/IEC/IEEE 41062:2023 Software engineering — Recommended practice for software acquisition
- ISO/IEC/IEEE 42010:2022 Software, systems and enterprise — Architecture description
- ISO/IEC/IEEE 42020:2019: Software, systems and enterprise — Architecture processes
- ISO/IEC/IEEE 42030: 2019 Software, systems, and enterprise — Architecture evaluation framework
- IEC 60300-1:2014 Dependability management — Part 1: Guidance for management and application.
- IEC/IEEE 82079-1 2019 Preparation of Information for Use (Instructions for Use) of Products — Part 1: Principles and General Requirements
- ISO/IEC/IEEE 90003:2018 Software engineering — Guidelines for the application of ISO 9001:2015 to computer software

CONSOLIDATED REFERENCE LIST

Appendix C

The Consolidated Reference List identifies all recommended reference materials (to the level of section number) that accompany the breakdown of topics within each knowledge area (KA). This Consolidated Reference List is adopted by the software engineering certification and associated professional development products offered by the IEEE Computer Society. KA Editors used the references allocated to their KA by the Consolidated Reference List as their Recommended References.

Collectively this Consolidated Reference List is

- Complete: Covering the entire scope of the SWEBOK Guide.
- Sufficient: Providing enough information to describe "generally accepted" knowledge.
- Consistent: Not providing contradictory knowledge nor conflicting practices.
- Credible: Recognized as providing expert treatment.
- Current: Treating the subject in a manner that is commensurate with currently generally accepted knowledge.
- Succinct: As short as possible (both in number of reference items and in total page count) without failing other objectives.

In total, there are 40 reference materials below.

- J.H. Allen et al., Software Security Engineering: A Guide for Project Managers, Addison-Wesley, 2008.
- L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 4th edition, 2021.
- M. Bishop, Computer Security: Art and Science, 2nd Edition, Addison-Wesley, 2018.
- B. Boehm and R. Turner, Balancing Agility and Discipline: A Guide for the Perplexed, Addison-Wesley, 2003.
- G. Booch, J. Rumbaugh and I. Jacobson, The Unified Modeling Language User Guide, 2nd edition, Addison-Wesley, 2005.
- F. Bott et al., Professional Issues in Software Engineering, 3rd ed., Taylor & Francis, 2000.
- F. Brooks, The Design of Design, Addison-Wesley, 2010
- J.G. Brookshear, Computer Science: An Overview, 12th ed., Addison-Wesley, 2017.
- D. Budgen, Software Design, 3rd ed., CRC Press, 2021.
- E.W. Cheney and D.R. Kincaid, Numerical Mathematics and Computing, 7th ed., Addison Wesley, 2020.
- P. Clements et al., Documenting Software Architectures: Views and Beyond, 2nd ed., Pearson Education, 2010.
- C. Dotson, Practical Cloud Security, O'Reilly, 2019.
- D. Farley, Modern Software Engineering: Doing What Works to Build Better Software Faster. Addison-Wesley Professional, 2022.
- R.E. Fairley, Managing and Leading Software Projects, Wiley-IEEE Computer Society Press, 2009.
- C.Y Laporte, A. April, Software Quality Assurance, IEEE Computer Society Press, 1st ed., 2018.
- E. Gamma et al., Design Patterns: Elements of Reusable Object-Oriented

Software, 1st ed., Addison-Wesley Professional, 1994.
- P. Grubb and A.A. Takang, Software Maintenance: Concepts and Practice, 2nd ed., World Scientific Publishing, 2003.
- A.M.J. Hass, Configuration Management Principles and Practices, 1st ed., Addison-Wesley, 2003.
- S.H. Kan, Metrics and Models in Software Quality Engineering, 2nd ed., Addison-Wesley, 2002.
- G. Kim, J. Humble, P. Debois, J. Willis and J. Allspaw, The DevOps handbook: How to create world-class agility, reliability, & security in technology organizations, 2nd ed., IT Revolution, 2021.
- M.W. Maier and E. Rechtin, The Art of Systems Architecting, 3rd edition, CRC Press, 2009.
- S. McConnell, Code Complete, 2nd ed., Microsoft Press, 2004.
- J. McGarry et al., Practical Software Measurement: Objective Information for Decision Makers, Addison-Wesley Professional, 2001.
- S.J. Mellor and M.J. Balcer, Executable UML: A Foundation for Model-Driven Architecture, 1st ed., Addison-Wesley, 2002.
- D. C. Montgomery and G. C. Runger, Applied Statistics and Probability for Engineers, 7th ed., Wiley, 2018.
- S. Naik and P. Tripathy, Software Testing and Quality Assurance: Theory and Practice, Wiley-Spektrum, 2008.
- J. Nielsen, Usability Engineering, 1st ed., Morgan Kaufmann, 1993.
- L. Null and J. Lobur, The Essentials of Computer Organization and Architecture, 5th ed. Jones and Bartlett Publishers, 2018.
- M. Page-Jones, Fundamentals of Object-Oriented Design in UML, 1st ed., Addison-Wesley, 1999.
- Project Management Institute and Agile Alliance, Agile Practice Guide, Project Management Institute, 2017.
- K. Rosen, Discrete Mathematics and its Applications, 8th ed., McGraw-Hill, 2018.
- N. Rozanski and E. Woods, Software Systems Architecture: Working with Stakeholders Using Viewpoints and Perspectives, 2nd edition, Addison-Wesley, 2011.
- J. Shore and S. Warden, The Art of Agile Development, O'Reilly Media, 2nd Edition, 2021.
- A. Silberschatz, P.B. Galvin, and G. Gagne, Operating System Concepts, 8th ed., Wiley, 2008.
- I. Sommerville, Software Engineering, 10th ed., Addison-Wesley, 2016.
- R.N. Taylor, N. Medvidović, E. Dashofy, Software Architecture: Foundations, Theory, and Practice, Wiley, 2009
- S. Tockey, Return on Software: Maximizing the Return on Your Software Investment, 1st ed., Addison-Wesley, 2004.
- G. Voland, Engineering by Design, 2nd ed., Prentice Hall, 2003.
- K.E. Wiegers, Software Requirements, 3rd ed., Microsoft Press, 2013.
- J.M. Wing, "A Specifier's Introduction to Formal Methods," Computer, vol. 23, no. 9, 1990, pp. 8, 10–23.

The Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), published by the IEEE Computer Society, represents the current state of generally accepted knowledge and promotes a consistent view of software engineering worldwide. Guide Version 4 reflects changes since the publication of Guide V3 in 2014, including modern development practices, new techniques, and the advancement of standards, such as areas and descriptions related to agile and DevOps, architecture, operations, security, and AI.

IEEE Computer Society is the largest computer science and technology community dedicated to engaging engineers, scientists, academia, and industry professionals from across the globe, driving continued advancements.

IEEE COMPUTER SOCIETY

IEEE